lsqvce 6.35.1 — User Manual

least-squares adjustment with variance component estimation

Contents

1 Introduction **3**
- 1.1 What it does for you . . . 3
- 1.2 Three lines . . . 4
- 1.3 Is it for you? . . . 4
- 1.4 The three documents . . . 4
- 1.5 How to read this Manual . . . 5

2 Installation **5**
- 2.1 Requirements . . . 5
- 2.2 Install . . . 5
- 2.3 Check . . . 5
- 2.4 Uninstall . . . 5
- 2.5 What is in the folder . . . 5

3 Key concepts **6**
- 3.1 The adjustment in one paragraph . . . 6
- 3.2 Three kinds of problem . . . 6
- 3.3 Parameters: common and local . . . 7
- 3.4 Data: systems, days, arcs and streams . . . 7
- 3.5 The two functions you write . . . 7
- 3.6 What `fit` does with them . . . 8
- 3.7 The noise model . . . 8
- 3.8 The result and the verdict . . . 8
- 3.9 Glossary . . . 9

4 Your first adjustment **9**
- 4.1 Part 1 — A straight line . . . 9
- 4.2 Part 2 — A streaming problem . . . 10
- 4.3 Where to go from here . . . 13

5 The workflow, step by step **13**
- 5.1 Step 1 — Decide which kind of problem you have . . . 14
- 5.2 Step 2 — List the parameters . . . 14
- 5.3 Step 3 — Write `buildDay` . . . 14
- 5.4 Step 4 — Share the expensive work of a day: `prepareDay` . . . 15
- 5.5 Step 5 — Declare the system and its noise . . . 16
- 5.6 Step 6 — First run: one solve under the declared model . . . 16
- 5.7 Step 7 — Estimate the stochastic model . . . 16
- 5.8 Step 8 — Add regularisation if the problem needs it . . . 16
- 5.9 Step 9 — Read the report and act on it . . . 16
- 5.10 Step 10 — Take the results out . . . 17

6 Declaring your problem **17**
6.1 The picture . . . 17
6.2 Choosing the scope of each parameter . . . 17
6.3 Writing `buildDay` with local parameters . . . 17
6.4 Arcs and schedules . . . 18
6.5 Several systems . . . 18
6.6 Sharing the work of a day: `prepareDay` . . . 18
6.7 Multi-axis observations and their accuracy . . . 19
6.8 A complete declaration: a GRACE-FO pair . . . 19
6.9 Checklist before the first run . . . 20

7 Choosing the stochastic model **20**
7.1 Three facts, three places . . . 21
7.2 What to declare . . . 21
7.3 Running the estimator . . . 21
7.4 Reading the estimate . . . 22
7.5 Outliers . . . 23

8 Priors and regularisation **24**
8.1 The idea . . . 24
8.2 Typical priors . . . 24
8.3 Writing `L` yourself . . . 24
8.4 Reading the result . . . 25
8.5 Activating a prior later . . . 25

9 Reading the results and the report **25**
9.1 The `Results` object . . . 25
9.2 How the report is built . . . 27
9.3 The verdict . . . 29
9.4 Level 0 — INTEGRITY (does the arithmetic hold?) . . . 29
9.5 Level 1 — GLOBAL (is the model consistent?) . . . 30
9.6 Level 2 — LOCALISE (where is it broken?) . . . 33
9.7 Level 3 — RELIABILITY (what can you not check?) . . . 36
9.8 Level 4 — PARAMETERS (what do you actually know?) . . . 38
9.9 Beyond the report: the stochastic model . . . 38
9.10 Adverse findings at a glance . . . 40
9.11 A worked reading . . . 41

10 Watching a run **43**
10.1 During the run: `OnPass` . . . 43
10.2 After the run: one figure per question . . . 43
10.3 Reproducing a run . . . 43

11 The examples **43**
11.1 Which example for which question . . . 44
11.2 Suggested reading order . . . 44
11.3 Examples, helpers and templates . . . 45
11.4 How to run them . . . 45
11.5 `demo_tutorial`: your first streaming run, with `prepareDay` . . . 46
11.6 `demo_workflow`: build your own case . . . 46
11.7 `demo_fit`: the three data kinds, one call each . . . 47
11.8 `demo_lsqvce`: six failure modes and their verdicts . . . 47
11.9 `demo_noisePolicy`: what you know about each stream . . . 48
11.10 `demo_priors`: regularisation in the announced layout . . . 49
11.11 `demo_combine`: several systems by elimination . . . 50
11.12 `demo_sensors`: system / sensor / day, gaps, and a bad day . . . 50

11.13 demo_grace and demo_grace_dataLoader: the noise loop on multi-axis streams 51
11.14 demo_gravity, demo_gravity_dataLoader and demo_gravity_design: a physically structured gravity recovery 52
11.15 demo_gravfire and template/gravfire_data.m: a GRAVFIRE bundle 53
11.16 template/kbr_variational.m: GRACE-FO range-rates by the variational equations . 55
11.17 template/lsqvce_template.m: the engine-tier starting point 56
11.18 demo_messaging: what each verbosity level prints 57
11.19 bench_qr_vs_normal: which backend, measured on your machine 57

12 Large problems **58**
12.1 Memory 58
12.2 Time 58
12.3 Resampling days 58
12.4 Backend 59
12.5 Before a long run 59

13 Troubleshooting **59**
13.1 Before any data is read: the declaration 59
13.2 While reading: your buildDay and prepareDay 60
13.3 During the solve 60
13.4 The run finishes, but the numbers look wrong 61
13.5 Asking for help 61

1 Introduction

lsqvce is a MATLAB toolbox for least-squares adjustment with variance component estimation. You describe your measurements and your unknowns; the toolbox estimates the unknowns, works out how noisy each kind of measurement really is, and tells you whether the result can be trusted.

It was built for satellite gravity recovery (GRACE / GRACE-FO). There, millions of correlated observations from several instruments determine a few thousand common parameters, together with many nuisance parameters per arc or per day. It works just as well on a small problem: a levelling network, a calibration, or a curve fit.

1.1 What it does for you

- **Solves the adjustment.** A linear or linearised Gauss–Markov model. The *common* parameters are shared by all data. The *local* parameters belong to one arc, one day or one instrument. They are eliminated on the fly, so the full design matrix is never formed and memory stays small.
- **Learns the noise from the data.** With `Estimate = "noise"` the toolbox estimates, for every observation stream, the spectral shape of its noise, its level, and a variance factor per arc. You only have to give a starting guess.
- **Regularises by declaration.** A prior (a Kaula rule, minimum constraints, a background model) is a group of pseudo-observations in your own columns. It is treated like data and can have its weight estimated.
- **Judges the result.** Global test, outlier tests, redundancy numbers, minimal detectable biases and the whiteness of every stream are reduced to one verdict by `R.report`.
- **Is robust on request.** M-estimation with Huber, IGG3 or bisquare weights, in the same loop as the noise estimation. Nonlinear models are solved by damped Gauss–Newton (the batched kind with `x0`); robust weights and Gauss–Newton do not combine yet.
- **Lets you watch.** A hook runs after every pass, and one plot answers each question afterwards.

1.2 Three lines

```
lsqvce_setup                                    % once, to install
R = lsqvce.fit(data, Estimate="noise");         % the one verb
R.report                                        % the verdict
```

`data` is a plain MATLAB struct that describes your problem. The options say how to run it. `R` holds everything that comes back. You never call anything else unless you want to. The toolbox has exactly six user-facing names:

name	what it is
`lsqvce.fit`	runs the adjustment: `R = lsqvce.fit(data, opts)`
`lsqvce.Options`	every setting of a run, with safe defaults
`lsqvce.Results`	what `fit` returns: estimates, errors, diagnostics, plots
`lsqvce.report`	prints the verdict (`R.report` calls it for you)
`lsqvce.layout`	shows the column layout `fit` will use, to write priors in
`lsqvce.solveContributions`	re-solves a fit for any weighting of its days (bootstrap)

1.3 Is it for you?

your situation	where to start
a design matrix `A`, observations `l` and weights that fit in memory	chapter *Your first adjustment*, part 1: three lines
a problem too large for memory, or a nonlinear model	chapter *The workflow, step by step*, step 1 (the *batched* kind)
data that arrive per day and per arc, from several instruments, with nuisance parameters and correlated noise	chapters *Key concepts* and *Your first adjustment*, part 2 (the *streaming* kind)

1.4 The three documents

document	answers	where
this Manual	*How do I use it?* Installation, concepts, a tutorial, the workflow, declaring your problem, the noise model, priors, reading the report, the examples	`doc lsqvce`, `lsqvce_manual.pdf`
the API reference	*What exactly does each interface accept and return?* The `data` struct, `buildDay` and `prepareDay`, covariance specifications, `Options`, `Results`, every error identifier	`html/lsqvce_api.html`, `lsqvce_api.pdf`
the theory	*What is computed, and why?* The mathematical framework	`lsqvce_theory.pdf`

Every function also has its own page: `doc lsqvce.<name>` or `help lsqvce.<name>`.

1.5 How to read this Manual

If you are new, read in this order:

1. *Installation* — five minutes.
2. *Key concepts* — the vocabulary every later chapter uses: common and local parameters, systems, days, arcs, streams, and the two functions you write (`buildDay` and, optionally, `prepareDay`).
3. *Your first adjustment* — a complete, runnable example (`examples/demo_tutorial.m`). Run it, then change it.
4. *The workflow, step by step* — the same path for your own data, with every decision explained.

When you write your own case, keep *Declaring your problem* and *Choosing the stochastic model* open. Come back to *Reading the results and the report* after every run. *Troubleshooting* lists the common error messages and what they mean.

For large campaigns, read *Large problems* before the first long run.

2 Installation

2.1 Requirements

MATLAB R2024a or later. The Parallel Computing Toolbox is optional (`Parallel.Mode = "parfor" | "spmd"`); without it the toolbox runs serially and says so once per run.

2.2 Install

```
cd <toolbox root>          % the folder holding +lsqvce
lsqvce_setup               % or lsqvce_setup('install')
```

Run this once, from the folder that holds `+lsqvce`. It adds exactly two path entries — the root with the `lsqvce.*` packages, and `examples/` — builds the Help browser index so that `doc lsqvce` opens this Manual, saves the path, and prints a check report. `template/` is not added: you copy a template into your own project.

2.3 Check

```
lsqvce_setup('check')      % version, path entries, doc index, whether lsqvce.fit resolves
```

Then run one example end to end. It takes a few seconds and needs no data of yours:

```
[R, s] = demo_tutorial();  % prints a report ending in "VERDICT: PASS"
```

If that prints the report, the installation works. The chapter *Your first adjustment* explains what you just ran.

2.4 Uninstall

```
lsqvce_setup('uninstall')  % removes the two path entries and the Help browser index;
                           % the folder itself is untouched
```

2.5 What is in the folder

folder / file	content
`+lsqvce/`	the package: the user tier (`fit`, `Options`, `Results`, `report`, `layout`, `solveContributions`) and the `engine`, `kernel` and `util` tiers below it
`examples/`	runnable examples (chapter *The examples*); start with `demo_tutorial`
`template/`	starting points for your own case: `kbr_variational.m` (GRACE-FO Level-1B range-rates and orbits), `gravfire_data.m` (a GRAVFIRE bundle to the `data` struct), `lsqvce_template.m`
`html/`	this Manual, the API reference and one page per public unit
`lsqvce_manual.pdf`, `lsqvce_api.pdf`, `lsqvce_theory.pdf`	the three documents as PDF
`RELEASE_NOTES.md`	what changed in each version
`dev/`	development: tests, documentation sources, the development record

3 Key concepts

This chapter introduces the vocabulary the rest of the Manual uses. It has no code to run. Read it once before the tutorial. The words in **bold** are the names you will meet as field names, options and chapter titles.

3.1 The adjustment in one paragraph

You have observations `l` that depend on unknown parameters `x` through a linear (or linearised) model, `l + v = A x`, where `v` are the errors. Least squares picks the `x` that makes the weighted errors smallest. The weights are the inverse of the noise covariance of the observations. Usually you do not know that covariance well. *Variance component estimation* (VCE) estimates it from the residuals, re-weights, and solves again until the weights settle. lsqvce does all of that in one call, `lsqvce.fit`.

3.2 Three kinds of problem

`fit` recognises the kind of your problem from the fields of `data`:

kind	fields you give	use it when
matrices	`A`, `l`, `p`	the whole design matrix fits in memory
batched	`batchFcn`, `numBatches`, `numUnknowns` (or `x0`)	it does not fit, or the model is nonlinear; your function returns one block of rows at a time
streaming	`commonParams`, `days`, `systems`	the data come per day and per arc, from several instruments, with nuisance parameters and correlated noise

The first two are ordinary least squares with diagnostics. The streaming kind is what the toolbox is built for, and the rest of this chapter describes it.

3.3 Parameters: common and local

Ask of every unknown: *over what span is it constant, and who sees it?*

- **Common parameters** (`data.commonParams`) are seen by all data, for example the gravity-field coefficients or station coordinates. They are what you want to know.
- **Local parameters** (`localParams` of a system) belong to one **arc**, one **day** or one **system**, for example an initial orbit state, an accelerometer bias, or a per-arc offset. They are nuisance parameters. The toolbox eliminates them arc by arc while it reads the data, so their number does not cost memory.

You declare both as tables with one row per *kind* of parameter. Column 1 is the scope (`'common'`, `'day'`, `'arc'` or `'system'`); the other columns are yours (a name, a degree and order, whatever helps you). The toolbox instantiates each row once per scope and counts the columns itself: you never compute a column index by hand.

3.4 Data: systems, days, arcs and streams

```
data
├── commonParams          what everybody estimates
├── days                  the calendar, e.g. 1:30
└── systems(k)            one per satellite, receiver, or instrument group
    ├── localParams       its nuisance parameters
    ├── covariance.<type>  one noise law per observation type  ──►  the streams
    ├── prepareDay        (optional) the work all its streams of a day share
    └── buildDay          the rows of one day and one observation type
```

- A **system** is one source of data with its own nuisance parameters, for example one satellite, or a satellite pair whose inter-satellite ranging depends on both orbits.
- A **day** is the unit of reading. The toolbox asks for the data of one day at a time.
- An **observation type** is a kind of measurement within a system, for example `'pos'` and `'kbr'`. The observation types are the field names of the system's `covariance` struct. Each (system, observation type) pair is one **stream**.
- An **arc** is one continuous piece of a stream on a day. The noise is correlated within an arc and independent between arcs. `buildDay` returns one cell per arc.

3.5 The two functions you write

You do not hand the toolbox your data. You hand it functions that produce the data on demand. This is what keeps memory small, but it has one consequence: **the toolbox calls your functions more than once for the same day**, so they must return the same result every time.

`buildDay` (required) returns the design rows and observations of one day for one observation type:

```
D = buildDay(day, obsType, common, local, procOpt, info)
%   D.A{a}  design of the common parameters, arc a      (m_a x nc)
%   D.B{a}  design of the local parameters,  arc a      (m_a x nl)
%   D.l{a}  observed minus computed                     (m_a x 1)
%   D.t{a}  epochs in seconds                           (m_a x 1)
```

`prepareDay` (optional) does the work that every stream of a day shares, *once* per day, before the first `buildDay` of that day. Its result reaches every `buildDay` of the day as `procOpt.prepared`:

```
P = prepareDay(day, common, local, procOpt, info)
```

Without `prepareDay`, a system with two observation types computes anything they share twice per day: an orbit integration, a large file read, a coordinate rotation. With it, the shared work happens once per build of the day. The tutorial uses it; chapter *Declaring your problem* shows when it pays.

Both functions take *as many arguments as they declare*. `@(day, obsType)` is a valid `buildDay` when you need nothing else. `info`, the 6th argument, tells you where each block of local parameters sits in `B`.

3.6 What `fit` does with them

For a run with `Estimate = "noise"`, `fit` repeats one **pass** until the noise settles:

```
for every pass
    for every system and every day
        P = prepareDay(day, ...)                  once per day (if declared)
        for every observation type
          D = buildDay(day, obsType, ...)      the day's rows, with procOpt.prepared = P
            whiten every arc with the stream's current noise law
            eliminate the local parameters of the arc
            add the arc to the normal equations of the common parameters
    solve for the common parameters
    read the days again: residuals, redundancies
    update the noise law of every stream, its level, and every arc factor
until no variance component changes by more than Tolerance
diagnose: outlier tests, reliability, whiteness  ->  R.report
```

With `Estimate = "none"` (the default) the loop runs once: one solve under the noise you declared. Each day is read at least twice per pass, once to accumulate and once for the residuals. `Options.CacheDays` decides how many built days are kept in memory between the two reads.

3.7 The noise model

Each stream has a **noise law** (`covariance.<type>`): how its errors correlate from epoch to epoch (`type`), and how large they are (`sigma`). On top of that, every arc can have its own **arc factor**, which catches one bad pass without spoiling the whole stream. With `Estimate = "noise"` the toolbox estimates all of them. You start from an honest guess, usually white noise (`type = 'diag'`) with a realistic `sigma`.

The result of all weights together is summarised by $\boldsymbol{\sigma_0}$ (`R.sigma0`), the a-posteriori standard deviation of unit weight. A σ_0 near 1 means the declared and the observed noise agree.

3.8 The result and the verdict

`fit` returns one `lsqvce.Results` object, `R`. It holds the estimate `R.x`, its covariance, the estimated noise model, the per-observation diagnostics, the history of the passes, and a record of how the run was made (`R.meta`). `R.report` reads all of this and prints a verdict in five levels:

level	question
0 integrity	does the arithmetic hold?
1 global	are model and noise consistent with the data?
2 localise	which observations are suspect?
3 reliability	which errors could go undetected?
4 parameters	how well is each parameter known?

The verdict is one of `PASS`, `PASS WITH WARNINGS`, `SUSPECT` and `INVALID` (see *Reading the results and the report*).

3.9 Glossary

term	meaning
arc	a continuous piece of one stream on one day; one cell of `buildDay`'s output
arc factor	a variance factor of one arc, estimated on top of the stream's level
common parameter	an unknown seen by all data (`data.commonParams`)
`Cep`	per-epoch accuracy delivered with the data (`D.Cep`), relative to the law's `sigma`
day	the unit in which the toolbox reads data
local parameter	a nuisance unknown of one arc, day or system, eliminated on the fly
observation type	a kind of measurement within a system; a field name of `covariance`
pass	one solve plus one update of the noise model
pool	streams that share one estimated spectral shape (`Options.Pool`)
prior	pseudo-observations on the parameters (`data.priors`): datum, regularisation
redundancy	how much of an observation is checked by the others (0 to 1)
σ_0	a-posteriori standard deviation of unit weight; about 1 when the weights are right
stream	one (system, observation type) pair, with one noise law
system	one source of data with its own local parameters (`data.systems(k)`)
whitening	transforming correlated errors into uncorrelated ones before the solve

4 Your first adjustment

This chapter has two parts. Part 1 fits a straight line in three lines of code. Part 2 builds a small streaming problem from scratch: two instruments, ten arcs, an unknown offset per arc, and noise whose colour is unknown. That is the shape of every real case the toolbox is built for. Part 2 is the file `examples/demo_tutorial.m`; run it, then read on.

4.1 Part 1 — A straight line

Copy this into the Command Window:

```
rng(1);
t     = (0:0.1:10).';
A     = [ones(size(t)), t];                % offset and slope
xTrue = [2; 0.5];
sigma = 0.1;                               % the noise of every observation
l     = A*xTrue + sigma*randn(size(t));    % simulated measurements

data = struct('A', A, 'l', l, 'p', ones(size(t)) / sigma^2);   % p are WEIGHTS: 1/sigma^2
R = lsqvce.fit(data);
R.report
R.coefficients()
```

`R.report` ends in `VERDICT: PASS`, and `R.coefficients()` prints

```
    index    estimate       se        lower      upper
      1       1.9845     0.019632     1.9461      2.023
      2       0.5022     0.003392    0.49555    0.50885
```

the estimate of offset and slope, its standard error, and a 95 % confidence interval. Both intervals contain the truth (2 and 0.5). `R.sigma0` is 0.994: the weights you gave match the scatter of the data.

Two things to remember from part 1:

- `p` holds **weights**, $1/\sigma^2$, not standard deviations.
- A σ_0 far from 1 means the weights are wrong in scale. Fix that before anything else.

If your problem fits in memory and the noise is known, this is all you need. The rest of the chapter is for data that do not fit this mould.

4.2 Part 2 — A streaming problem

4.2.1 The case

A platform observes a periodic signal of one hour period,

$$y(t) = a\cdot\cos(2\pi t / 3600) + b\cdot\sin(2\pi t / 3600),$$

with two instruments on one clock, a **gauge** and a **radar**. It records two arcs a day of two hours each (240 epochs, 30 s apart) on five days. Every arc starts with an unknown offset that both instruments see. The gauge has white noise with $\sigma = 0.05$. The radar has correlated noise (AR(1), $\varphi = 0.5$) with $\sigma = 0.02$, but we pretend we do not know that and declare it white.

what	here	in general
common parameters	`a`, `b`	what you want to know
local parameters	one offset per arc (10 in all)	nuisance parameters, eliminated
system	the platform	one satellite, receiver, instrument group
observation types	`'gauge'`, `'radar'`	the kinds of measurement
what both streams share per day	the epochs and the signal basis	an orbit, a file, a rotation

4.2.2 Step 1 — Declare the parameters and the calendar

```
data = struct();
data.commonParams = {'common', 'cosAmp'; 'common', 'sinAmp'};
data.days = 1:5;
```

One row per parameter. Column 1 is the scope; column 2 is a name of your choice.

4.2.3 Step 2 — Declare the system

```
sys = struct();
sys.name        = 'platform';
sys.localParams = {'arc', 'bias'};             % one offset per arc
sys.arcs        = 2;                           % arcs 1 and 2 on every day
sys.processingOptions = struct('epochs', 240, 'step', 30, ...
                               'truth', [0.8; -0.3]);
sys.prepareDay  = @prepareTutorialDay;         % once per day
```

```
sys.buildDay    = @buildTutorialDay;           % once per day and observation type
sys.covariance.gauge = struct('type', 'diag', 'sigma', 0.05);
sys.covariance.radar = struct('type', 'diag', 'sigma', 0.02);   % colour unknown
data.systems = sys;
```

Line by line:

- `localParams = {'arc', 'bias'}` asks for one parameter per arc. The toolbox counts them (5 days × 2 arcs = 10) and tells `buildDay` where each one sits.
- `arcs = 2` is the arc schedule: arcs 1 and 2 on every day. A system without `'arc'` parameters may leave it out (its arcs are then simply the cells `buildDay` returns). With `'arc'` parameters it is required, because their number must be known before any data is read; leaving it out is an error that says so.
- `processingOptions` is a struct of your own. The toolbox passes it unchanged to your functions as `procOpt`. Here it carries the simulator's settings. In a real case it carries a data directory, a model file, a reference frame.
- The **field names of `covariance` are the observation types**. There is no separate list. Each gets a noise law; `'diag'` is white noise of level `sigma`.

4.2.4 Step 3 — Write `prepareDay`: what both instruments share

Both streams need the epochs of every arc and the signal basis at those epochs. Computing this in `buildDay` would do it twice per day. `prepareDay` does it once:

```
function P = prepareTutorialDay(day, ~, ~, procOpt, info)
    nArc = numel(info.arcId);
    P = struct('t', {cell(1, nArc)}, 'G', {cell(1, nArc)}, 'y', {cell(1, nArc)});
    for a = 1:nArc
        start  = (day - 1) * 86400 + (info.arcId(a) - 1) * 43200;   % arcs at 0 h and 12 h
        t      = start + (0:procOpt.epochs - 1).' * procOpt.step;
        w      = 2 * pi / 3600;
        G      = [cos(w * t), sin(w * t)];
        rs     = RandStream('mt19937ar', 'Seed', 100 * day + info.arcId(a));
        bias   = 0.5 * randn(rs);                  % this arc's true offset
        P.t{a} = t;
        P.G{a} = G;
        P.y{a} = G * procOpt.truth + bias;
    end
end
```

It receives the day, the two parameter tables (unused here, hence `~`), your `processingOptions` and `info`. `info.arcId` lists the arcs of this day. Whatever it returns, `P`, is handed to every `buildDay` of the same day.

4.2.5 Step 4 — Write `buildDay`: the rows of one instrument

```
function D = buildTutorialDay(day, obsType, ~, ~, procOpt, info)
    P    = procOpt.prepared;                    % what prepareTutorialDay returned
    nArc = numel(info.arcId);
    D = struct('A', {cell(1, nArc)}, 'B', {cell(1, nArc)}, ...
               'l', {cell(1, nArc)}, 't', {cell(1, nArc)});
    for a = 1:nArc
        m  = numel(P.t{a});
        rs = RandStream('mt19937ar', 'Seed', 1000 * day + 10 * info.arcId(a) + double(obsType(1)))
        switch obsType
            case 'gauge'
                e = 0.05 * randn(rs, m, 1);                          % white
```

```
            case 'radar'
                phi = 0.5;                                              % AR(1)
                e = filter(1, [1 -phi], 0.02 * sqrt(1 - phi^2) * randn(rs, m + 200, 1));
                e = e(201:end);
        end
        B = zeros(m, info.nl);
        B(:, info.offsetOf(info.arcId(a)) + (1:info.count(a))) = 1;   % this arc's offset
        D.A{a} = P.G{a};                                % m x 2: the common parameters
        D.B{a} = B;                                     % m x 10: the local parameters
        D.l{a} = P.y{a} + e;                            % observed minus computed
        D.t{a} = P.t{a};                                % epochs in seconds
    end
end
```

The four rules every `buildDay` follows:

1. **One cell per arc** in each of `A`, `B`, `l`, `t`, all with the same number of rows.
2. **Columns by `info`, never by hand.** `info.nl` is the total number of local parameters; `info.offsetOf(arcId)` is where this arc's block starts; `info.count(a)` is its width.
3. **Raw rows.** `l` is observed minus computed, unweighted and unwhitened. The toolbox applies the noise law itself.
4. **Deterministic.** The toolbox reads every day more than once. Here the random noise comes from a stream seeded on (day, arc, instrument), so every call returns the same rows. Your file reader is deterministic anyway.

Because `buildDay` uses `procOpt` (the 5th argument), it must declare at least five arguments; the toolbox refuses a `prepareDay` whose result could never arrive.

4.2.6 Step 5 — A first run under the declared noise

```
R1 = lsqvce.fit(data);
R1.coefficients()
```

`Estimate = "none"` is the default: one solve with the weights you declared. Always run this first. It shows that the declaration is accepted (every declaration error names the field at fault) and that σ_0 is plausible. Here $\sigma_0 = 0.993$.

4.2.7 Step 6 — Estimate the noise

```
opts = lsqvce.Options(Estimate="noise", Seed=1);
R = lsqvce.fit(data, opts);
R.report
```

The loop solves, looks at the residuals of every stream, updates each stream's noise law and arc factors, and solves again. It stops when nothing changes by more than `Tolerance`; here after four passes (`R.stopReason` is `"converged"`). `Seed` makes the random probes that estimate the redundancies reproducible.

The report ends in `VERDICT: PASS`. Its Level 1 says *"The stochastic model settled in 4 pass(es)"* and *"Global test PASSES"*.

4.2.8 Step 7 — Compare

`demo_tutorial` prints:

```
param       truth       declared noise      estimated noise
cosAmp     0.8000     0.8011 +- 0.0005     0.8012 +- 0.0008
sinAmp    -0.3000    -0.3006 +- 0.0005    -0.3004 +- 0.0008
sigma0                           0.993                1.000   (converged)
```

The estimates hardly move. What moves is **how well you know them**. In the first run, σ_0 was fine, because the radar's *level* was declared right. But its *colour* was not: correlated errors carry less information than white ones, so the first run's standard errors are about 1.6 times too small. After the noise estimation they are honest. This is the most common way an adjustment goes wrong without any error message, and the reason `Estimate = "noise"` exists.

4.2.9 Step 8 — Look

```
R.summary()              % one row per stream: its final level relative to the declared one
R.plot("psd")            % the estimated noise spectrum of each stream
R.plot("residuals")      % the whitened residuals per stream
```

`R.summary()` lists the streams as `x1|gauge` and `x1|radar`: `x1` is the system's id, given automatically because we declared none (set `sys.id` to choose one). The radar's spectrum is no longer flat: the estimator found the colour we left out.

4.3 Where to go from here

- Replace the simulator in `prepareTutorialDay` and `buildTutorialDay` with your own reader, and the two `commonParams` rows with your parameters. *The workflow, step by step* goes through the decisions in order.
- *Declaring your problem* covers several systems, day and system scopes, data gaps, multi-axis observations and per-epoch accuracies.
- *Choosing the stochastic model* says which noise law to start from and how to read the estimate.
- *Reading the results and the report* explains every line of the report.

5 The workflow, step by step

This chapter is the checklist for your own data: the decisions in the order you make them, each with the chapter that treats it in depth. If you have not yet run the tutorial (*Your first adjustment*), do that first; it is the same path on a small case. `examples/demo_workflow.m` walks the path as a script to run section by section.

step	you decide	you write
1	which kind of problem	—
2	the parameters and their scopes	`commonParams`, `localParams`
3	how one day of one observation type becomes rows	`buildDay`
4	whether the streams of a day share expensive work	`prepareDay` (optional)
5	the systems and their starting noise	`data.systems`, `covariance`
6	— run once under the declared noise	
7	— estimate the noise	`lsqvce.Options`
8	regularisation, if needed	`data.priors`
9	— read the report and act on it	
10	— take the results out	

5.1 Step 1 — Decide which kind of problem you have

your problem	kind	you supply
the design matrix fits in memory, one block of unknowns	**matrices**	`data.A`, `data.l`, `data.p`
too large for memory, or nonlinear, one block of unknowns	**batched**	`data.batchFcn`, `data.numBatches`, `data.numUnknowns` (or `data.x0`)
data arrive per day and arc, several instruments, nuisance parameters per arc or day, correlated noise	**streaming**	`data.commonParams`, `data.days`, `data.systems` with your `buildDay`

The first two are complete after one line:

```
data = struct('A', A, 'l', l, 'p', 1 ./ sigma.^2);     % weights, not sigmas
R = lsqvce.fit(data);
R.report
```

The rest of this chapter is the streaming kind, the one the toolbox is built for.

5.2 Step 2 — List the parameters

Write two tables. `data.commonParams` holds the parameters every observation can see (for gravity recovery: the spherical-harmonic coefficients); `localParams` of a system holds the nuisance parameters that live on one arc, one day or the whole system (initial states, accelerometer calibration, empirical terms). Column 1 of every row says how often it is instantiated; the remaining columns are yours:

```
data.commonParams = [repmat({'common'}, nc, 1), myCoefficientNames];   % nc rows
s.localParams = [ {'arc', 'bias'}; {'arc', 'drift'}; {'day', 'accScale'} ];
```

The toolbox derives every count and every column position from these tables. See *Declaring your problem*.

5.3 Step 3 — Write `buildDay`

One function returns the data of one day and one observation type, one cell per arc:

```
function D = myBuildDay(day, obsType, common, ~, procOpt)
    arcs = readMyArcs(procOpt.dataDir, day, obsType);   % your reader, deterministic
    n = numel(arcs);
    D = struct('A', {cell(1, n)}, 'B', {cell(1, n)}, 'l', {cell(1, n)}, 't', {cell(1, n)});
    for a = 1:n
        m = numel(arcs(a).omc);
        D.A{a} = myPartials(arcs(a), common);   % m x nc
        D.B{a} = zeros(m, 0);                   % no local parameters in this sketch
        D.l{a} = arcs(a).omc;                   % observed minus computed
        D.t{a} = arcs(a).tSeconds;              % epochs in seconds
    end
end
```

Return raw, unwhitened rows; never scale them by your weights. With local parameters, add a 6th argument `info` and place every column by its offsets (*Declaring your problem*).

The six arguments, in plain words. Declare only as many as you use; the toolbox passes as many as your function declares (`@(day, obsType)` is fine):

argument	what you get
`day`	the day to build, one entry of `data.days` (or of the system's own `days`)
`obsType`	the observation type to build, one field name of `covariance`, as a char
`common`	your `data.commonParams` table, as you declared it
`local`	the system's `localParams` table, as you declared it
`procOpt`	the system's `processingOptions`, unchanged; with a `prepareDay`, also `procOpt.prepared`
`info`	where each block of parameters sits on this day: `info.nc`, `info.nl`, `info.arcId`, `info.offsetOf(arcId)`, `info.count`, `info.dayOffset`, … (API reference, chapter `buildDay`)

`buildDay` must be **deterministic**: the toolbox calls it at least twice per day and pass, and must get the same rows back each time.

5.4 Step 4 — Share the expensive work of a day: `prepareDay`

`buildDay` is called once per day **and observation type**. If your system has several observation types that rest on one expensive computation — an orbit integration with its partials, a large file both streams read, an Earth-rotation matrix for every epoch — that computation would run once per type. Move it into the system's `prepareDay`:

```
s.prepareDay = @myPrepareDay;              % P = myPrepareDay(day, common, local, procOpt, info)
s.buildDay   = @myBuildDay;                % reads procOpt.prepared

function P = myPrepareDay(day, ~, ~, procOpt, info)
    P.orbit = integrateOrbit(procOpt.dataDir, day, info);    % the expensive part, once
end

function D = myBuildDay(day, obsType, common, ~, procOpt, info)
    P = procOpt.prepared;                  % what myPrepareDay returned for this day
    switch obsType
        case 'pos', D = positionRows(P.orbit, day, info);
        case 'kbr', D = rangeRateRows(P.orbit, day, info);
    end
end
```

What to know:

- It runs **once per build of a day**, before the first `buildDay` of that day, and its result `P` reaches every `buildDay` of that day as `procOpt.prepared`.
- Its arguments are `(day, common, local, procOpt, info)`; declare as many as you use, none to all five.
- `buildDay` must then declare at least five arguments, or it could never see `procOpt.prepared` (refused as `lsqvce:makeLoader:prepareUnused`). `processingOptions` must not have a field named `prepared` of its own (`lsqvce:makeLoader:preparedField`). Through `fit`, both arrive under its own prefix (`ME.identifier` ends in `:prepareUnused` or `:preparedField`), the original attached as the cause.

- Only the current day's result is kept. A day that is built again (the default `CacheDays = 1` builds each day twice per pass) runs `prepareDay` again, so it must be deterministic too. `CacheDays = Inf` builds every day only once per run, if memory allows.
- If it throws, the error is `lsqvce:makeLoader:prepareDay`, naming the day, with your error attached as the cause.

A system with one observation type gains nothing from `prepareDay`; leave it out. The tutorial (*Your first adjustment*) and `template/kbr_variational.m` use it.

5.5 Step 5 — Declare the system and its noise

```
s = struct();
s.buildDay = @myBuildDay;
s.covariance.range = struct('type', 'diag', 'sigma', 2e-7);   % white noise, level 2e-7
s.processingOptions = struct('dataDir', '/data/mission');
data.days = 1:30;
data.systems = s;
```

The fieldnames of `covariance` are your observation types. Start with a `diag` law and a realistic level; the estimator can find the colour (*Choosing the stochastic model*).

5.6 Step 6 — First run: one solve under the declared model

```
R = lsqvce.fit(data);          % Estimate = "none"
R.report
```

Check two things before anything else: that the run completes without a declaration error (every error names the field at fault), and that `R.sigma0` is plausible. A σ_0 far from 1 means the declared accuracies are wrong in scale — fix that before estimating anything on top of it.

5.7 Step 7 — Estimate the stochastic model

```
opts = lsqvce.Options(Estimate="noise", Pool="stream", Seed=1);
R = lsqvce.fit(data, opts);
R.report
R.plot("psd"); R.plot("convergence");
```

The loop re-solves until the noise laws settle. `Pool` decides which streams share one spectral shape; `ArcFactors` (default on) gives every arc its own variance factor; `Seed` makes the probe-based redundancies reproducible.

5.8 Step 8 — Add regularisation if the problem needs it

A free network needs a datum; a high-degree gravity field needs a Kaula rule. Both are declared as priors in your columns:

```
data.priors(1) = struct('group', '^C|^S', 'x0', 0, 'sigma', 1e-10, 'estimate', "scale");
```

See *Priors and regularisation*.

5.9 Step 9 — Read the report and act on it

`R.report` prints a verdict and the evidence behind it: the global test, flagged observations, the reliability, the whiteness of every stream. Each adverse verdict has a standard next step — snooping, a richer noise law, a functional model correction. See *Reading the results and the report*.

5.10 Step 10 — Take the results out

```
T = R.coefficients();                % estimates, formal errors, confidence bounds
[C, L] = R.toTable(data);            % your own tables with estimates written in
save('run.mat', 'R');                % R.meta records version, seeds and options
```

6 Declaring your problem

This chapter shows how to map a real problem onto the `data` struct. The field-by-field contract — every field, its type, its default and every error — is in the API reference (chapters *The `data` struct* and *`buildDay`*).

6.1 The picture

You describe one adjustment with three answers: *what is estimated for everybody* (`data.commonParams`), *which days are processed* (`data.days`), and *which systems deliver data* (`data.systems`, one entry per satellite, receiver or instrument). Each system points to one function you write, `buildDay`, that returns the design rows and observations of one day for one observation type. Everything else — reading the days in order, whitening every arc with its noise law, eliminating the local parameters, iterating the variance components — is done by the toolbox.

The design is block-arrow shaped: every arc sees the common parameters through its `A` and its own local parameters through its `B`. The local blocks are eliminated arc by arc, so the memory need is set by the number of common parameters, not by the number of observations.

6.2 Choosing the scope of each parameter

Ask of every parameter: *over what span is it constant?*

constant over …	table	token	examples
the whole adjustment, seen by all systems	`commonParams`	`'common'`	gravity-field coefficients, station coordinates
one calendar day, seen by all systems	`commonParams`	`'day'`	daily common terms
one arc of one system	`localParams`	`'arc'`	initial position and velocity, per-arc biases
one day of one system	`localParams`	`'day'`	accelerometer scale and bias
the whole adjustment, one system	`localParams`	`'system'`	antenna offsets, instrument constants

Rows are **templates**: the toolbox instantiates every row once per scope and derives the column counts. With `data.paramColumns` a row can also carry a validity interval and is then instantiated only on the days it covers — a one-day manoeuvre parameter is just a row with a one-day validity.

6.3 Writing `buildDay` with local parameters

Declare the 6th argument and never compute a column offset yourself:

```
function D = buildDayGrace(day, obsType, common, ~, procOpt, info)
    raw  = readGraceDay(procOpt.dataDir, day, obsType);  % your reader, deterministic
    nArc = numel(info.arcId);                            % the schedule's arcs, in order
```

```
    D = struct('A', {cell(1, nArc)}, 'B', {cell(1, nArc)}, ...
               'l', {cell(1, nArc)}, 't', {cell(1, nArc)});
    for a = 1:nArc
        arc = raw.arc(a);                                % empty on a data gap
        D.A{a} = shPartials(arc, common);                % m_a x info.nc
        D.l{a} = arc.observed - arc.computed;            % observed minus computed
        D.t{a} = arc.tSeconds;
        B = sparse(numel(D.l{a}), info.nl);
        B(:, info.offsetOf(info.arcId(a)) + (1:info.count(a))) = arcPartials(arc);
        if info.dayCount > 0                             % this day's 'day' block
            B(:, info.dayOffset + (1:info.dayCount)) = accPartials(arc);
        end
        D.B{a} = B;
    end
end
```

`B` may be sparse; it usually is, because an arc touches only its own block.

6.4 Arcs and schedules

The cells `buildDay` returns **are** the arcs of the day. Without an `arcs` field the toolbox takes them as they come, and days may differ in their number of arcs. Declare a schedule only when `'arc'` parameters need to be counted before any data is read:

```
s.arcs = 4;                                  % arcs 1..4 on every day
s.arcs = @(day, obsType) arcTable{day};      % a resolver for ragged schedules
```

A data gap is an empty cell; a day without any data in a system belongs off that system's calendar (`s.days`).

6.5 Several systems

Each satellite or instrument is one entry of `data.systems`, with its own `buildDay`, covariance specs, local parameters and calendar. Systems share only the common parameters. An observation that depends on the local parameters of two satellites — the range-rate between two GRACE spacecraft — belongs to **one** system whose local table holds both satellites' parameters. One reader for several setups takes the setup as data: plain values in `processingOptions`, its 5th argument, or a closure that binds a configuration (API reference, chapter `buildDay`, *Where the handle goes*).

6.6 Sharing the work of a day: `prepareDay`

`buildDay` is called once per day **and observation type**. When several streams of a system rest on one expensive computation — the orbits of both satellites and their partials for the whole day, used by the kinematic orbits *and* by the range-rates — that computation would run once per stream. Give the system a `prepareDay` instead. It runs **once per build of a day**, before the first stream of the day is built, and every `buildDay` of that day finds its result in `procOpt.prepared`.

```
s.prepareDay = @myPrepareDay;    % P = myPrepareDay(day, common, local, procOpt, info)
s.buildDay   = @myBuildDay;      % D = myBuildDay(day, obsType, common, local, procOpt, info)
```

question	answer
when does it pay?	when two or more observation types of one system share an expensive step: an orbit integration, the variational partials, a large file, Earth-rotation matrices. With one observation type it gains nothing
what does it get?	as many of `(day, common, local, procOpt, info)` as it declares; `info` is the same day slice `buildDay` gets
what may it return?	anything: a struct, a cell, an object. The toolbox does not look inside
what must `buildDay` do?	declare at least five arguments, so that it receives `procOpt` (otherwise `lsqvce:makeLoader:prepareUnused`, reported by `fit` under its own prefix), and read `procOpt.prepared`
how often does it run?	once each time the day is built: twice per pass with the default `CacheDays = 1`, once per run with `CacheDays = Inf`, and on every parallel worker that builds the day
how much memory?	the current day's result only; it is dropped when the next day is prepared
what must it be?	deterministic, like `buildDay`
what if it fails?	`lsqvce:makeLoader:prepareDay`, naming the day, with your error as the cause

The tutorial (*Your first adjustment*, `examples/demo_tutorial.m`) is the smallest working example; the GRACE-FO pair below and `template/kbr_variational.m` are the real case. The full contract is in the API reference, chapter `buildDay`, *What all streams of a day share*.

6.7 Multi-axis observations and their accuracy

A stream with several axes per epoch — a kinematic orbit, a gradiometer — is returned epoch-major (x_1 y_1 z_1 x_2 ...). What the preprocessing knows about the accuracy of every epoch travels with the day as `D.Cep`, one cell per arc:

```
D.Cep{a} = sigmaPerEpoch;                                % n x 1   scalar stream
D.Cep{a} = [sx sy sz];                                   % n x 3   per-axis sigmas
D.Cep{a} = permute(reshape(Q.', 3, 3, []), [3 1 2]);     % n x 3 x 3 full covariance
```

The vector forms are standard deviations and the matrix form is a covariance — check one epoch by hand before a campaign. `Cep` is relative to the `sigma` of the covariance spec: declare `sigma = 1` when `Cep` already carries the absolute accuracy.

6.8 A complete declaration: a GRACE-FO pair

The range-rate between the two spacecraft depends on both orbits, so the pair is **one** system with three observation types: the kinematic positions of each satellite and the inter-satellite range-rate. All three rest on the same orbit integration, which therefore goes into `prepareDay`:

```
data.commonParams = [repmat({'common'}, nSh, 1), shParameterTable(60)];
data.days = 1:31;

state = {'x0'; 'y0'; 'z0'; 'vx0'; 'vy0'; 'vz0'};
s = struct();
```

```
s.id = 'G';  s.name = 'GRACE-FO';
s.prepareDay = @prepareGracePair;          % both orbits and their partials, once per day
s.buildDay   = @buildDayGracePair;         % picks the rows of one observation type
s.covariance.posC = struct('type', 'diag', 'sigma', 0.02);
s.covariance.posD = struct('type', 'diag', 'sigma', 0.02);
s.covariance.kbr  = struct('type', 'diag', 'sigma', 1e-7);   % colour: let the estimator find it
s.arcs = @(day, obsType) arcSchedule(day);                    % 'arc' rows need a schedule
s.localParams = [ ...
    [repmat({'arc'}, 6, 1), strcat(state, 'C')];                        % initial state C
    [repmat({'arc'}, 6, 1), strcat(state, 'D')];                        % initial state D
    {'arc', 'kbrBias'; 'arc', 'kbrCos1cpr'};                            % KBR empiricals
    [repmat({'day'}, 6, 1), {'accScaleC'; 'accBiasC'; 'accDriftC'; ...
                             'accScaleD'; 'accBiasD'; 'accDriftD'}]];  % ACC per day
s.processingOptions = struct('dataDir', '/data/gracefo');
data.systems = s;

R = lsqvce.fit(data, Estimate="noise");
R.report
```

with the two functions of the system:

```
function P = prepareGracePair(day, ~, ~, procOpt, info)
    P.C = integrateWithPartials('C', day, procOpt, info);   % r, v and d[r;v]/dp
    P.D = integrateWithPartials('D', day, procOpt, info);
end

function D = buildDayGracePair(day, obsType, ~, ~, procOpt, info)
    P = procOpt.prepared;                  % prepareGracePair's result for this day
    switch obsType
        case 'posC', D = kinematicRows(P.C, day, info);
        case 'posD', D = kinematicRows(P.D, day, info);
        case 'kbr',  D = rangeRateRows(P.C, P.D, day, info);
    end
end
```

A runnable builder of this shape for real Level-1B data is `template/kbr_variational.m` (chapter *The examples*).

6.9 Checklist before the first run

- `buildDay` returns `A`, `B`, `l`, `t` as one cell per arc, each arc with one row count; `l` is observed minus computed; nothing is whitened.
- Calling `buildDay` twice with the same arguments gives identical output.
- `t` is in seconds.
- `size(A, 2)` equals the derived nc, `size(B, 2)` the derived nl — build them from `info.nc` / `info.nl` and place blocks by the `info` offsets.
- Every observation type is a field of `covariance`.
- `Cep`, if given, has one cell per arc and matches the rows (n·d = m).
- No unknown field names in `data`: a typo is an error on purpose.

7 Choosing the stochastic model

The weights of a least-squares solution are the inverse of the noise covariance. This chapter says what to declare, what to let the data estimate, and how to read the estimate back. The laws and their fields are listed in the API reference (*Covariance specifications*); the mathematics is in the theory.

7.1 Three facts, three places

fact	declared in	estimated?
the **temporal law** — how the errors of a stream correlate from epoch to epoch	`covariance.<type>.type` and its parameters	yes, with `Estimate = "noise"`
the **level** of that law	`covariance.<type>.sigma` (a standard deviation)	yes
the **per-epoch accuracy** — what the preprocessing knows about each observation	`D.Cep`, returned by `buildDay`	never: it is data

The toolbox combines them per arc. A filter law (`diag`, `arma`, an AR-whitened `psd`, `var`) is applied in batches of up to 20 000 rows that overlap by the memory of the filter (for an `arma` law with an MA part, until its impulse response has decayed to rounding), so the batching does not change the result. A dense law (`acf`, `custom`, a `psd` whitened as Toeplitz) is factorised over the whole arc: exact, at a cost of $O(m^3)$ and m^2 memory for an arc of m rows — for arcs of many thousand epochs prefer a filter law. Above 20 000 epochs a dense law is refused (`lsqvce:covModel:denseArcTooLong`, 3.2 GB for the factor alone at that length); the field `maxDenseArc` on the covariance spec raises the limit deliberately. Under `Estimate = "noise"` the estimator's sweep forms every arc's covariance densely whatever the whitener, so there the same limit applies to every law, a filter law included.

7.2 What to declare

- **You do not know the colour of the noise.** Declare `type = 'diag'` with a realistic `sigma` and run `Estimate = "noise"`: the estimator starts from white noise and finds the spectral shape. Under `estimate = "law"` (the default) this is what happens to every stream, whatever law it declares: the declared law contributes only its `sigma`, as the starting level, and the shape is estimated from white noise. (For an `arma` law `sigma` is an innovation standard deviation; as a level it is read as the marginal one.)
- **You know the noise spectrum.** Declare `type = 'psd'` with `f` and `S`, or an `arma` filter; set `estimate = "scale"` to keep the shape and estimate only the level, or `"none"` to use it as given.
- **Your axes are correlated in time across axes.** Declare a `var` law (no `Cep`). It is used as declared: `Estimate = "noise"` has no path for it and refuses it (`lsqvce:fit:varUnderNoise`).
- **A stream you trust completely** — a prior stream, a reference — takes `estimate = "none"`.

7.3 Running the estimator

```
opts = lsqvce.Options(Estimate="noise", Pool="obsType", ArcFactors=true, ...
                      MaxPasses=7, Seed=1, Noise=struct('Grid', 120, 'Whitener', "ar"));
R = lsqvce.fit(data, opts);
```

Per pass the toolbox solves under the current laws, sweeps every arc once for its whitened residuals and their redundancies, and updates the spectral shape of every pool group, the level of every stream, the factor of every arc and the σ of every prior with `estimate = "scale"`. It stops when no component changes by more than `Tolerance` — the σ of a prior group included, which is first updated on the pass after it enters the solve, so the loop does not stop before — or after `MaxPasses`. A spectral bin the data barely determine may change by more, in proportion to its own standard error (chapter *Reading the results and the report*, "When the loop did not settle").

The choices that matter:

choice	guidance
`Pool`	the default `"stream"` gives every stream its own shape. Streams of the same physics may share one (`"obsType"`: both satellites' range-rate), but measured on identical spectra that gains nothing for x. Never pool streams of different physics or units: positions with range-rate drives the normal matrix towards singularity, and a shared law biases both levels (`poolNotWhite` says so)
`ArcFactors`	on when arcs are numerous and long enough to carry their own variance; off for few or short arcs
`Noise.Grid`	the spectral resolution of the estimated law; finer grids resolve narrow spectral features but give each bin less redundancy per pass. Keep 60 with the default `Smooth = 1`. Fewer bins cut the estimator's variance too, but bias the level low when the spectrum has a sharp low-frequency peak (16 bins: −12 % at AR(0.98))
`Noise.Whitener`	how every `psd` law — the estimated one and any you declare — whitens an arc: `"ar"` for long arcs (cost O(m·p) instead of O(m^3) in the solve); `"toeplitz"` (the default, exact) when the spectrum has narrow features a low-order filter cannot follow. Under `Estimate = "noise" "ar"` saves the solve only — the sweep forms each arc's covariance densely (above) — and the estimated spectrum is then whitened by an AR(30) that approximates it: exact enough for short-memory noise (an AR(1) law to about 1 %), biased where AR(30) cannot follow the shape (a 1/f floor: tens of percent per bin, and the loop need not settle)
`Probes`, `Seed`	the redundancies are estimated with random probes: 100 probes are the standard; a fixed `Seed` makes the run reproducible

Run one pass with `Estimate = "none"` first. If σ_0 is near 1 the declared weights are consistent. A σ_0 far from 1 says the `Cep` scale is wrong — most often an information matrix used where a covariance was expected — and estimating a law on top will not fix it.

7.4 Reading the estimate

```
R.noise.stream           % the stream keys, in the order of model{}
R.noise.model{k}.sigma   % the estimated level of stream k
R.noise.Sgroups          % the estimated spectrum of each pool group
R.summary()              % one row per stream: level, whiteness
R.plot("psd")            % the spectra
```

The standard deviation of one arc is the product of three terms: the declared `sigma`, the square root of the arc factor, and `R.sigma0`. With `sigma = 1` and a `Cep` carrying the absolute accuracy, that product reads directly as *how much too optimistic the preprocessing's covariance was on*

this arc.

After convergence the whitened residuals of every stream should be white. The loop tests that in every pass, per stream, with a Ljung–Box test over 20 lags at $\alpha = 0.05$; the result is in `R.summary()` and `R.history.white` (it is not a stop criterion). It detects short-lag correlation, periodic terms and excess low-frequency noise; a correlation that exists only at one long lag (an echo, a repeat-track pattern) is invisible to it — test for that with `R.systematic('Lags', L)`.

Three assumptions of the model can be contradicted by the data, and the loop checks them after convergence (`R.noise.checks`, one warning each):

warning	what the data say	what to do
`lsqvce:adjustVCEjoint:poolNotWhite`	a stream of a pool group is not white under the group's shared law. Either the group mixes spectra, which biases both levels and the formal errors, or the stream has something unmodelled in it (an event, a drift)	compare `Pool = "stream"`; regroup, or find the event
`lsqvce:adjustVCEjoint:levelVariable`	a stream's arcs scatter far beyond their sampling error around one level: the level changes from arc to arc or day to day	arc factors for that stream
`lsqvce:adjustVCEjoint:memoryVsArc`	the estimated correlation time exceeds half the median arc: the arcs' local parameters absorb the long memory, and the level comes out low (−12 % at a correlation time of 200 against 256-epoch arcs)	longer arcs, or a law that models the low frequencies

The formal errors of an estimated noise model are *plug-in*: `R.sol.Cxx` treats the estimated covariance as known. On short or few arcs they are 3–13 % optimistic in σ; on long records the optimism is negligible.

Arcs can be skipped in a pass: `gap` (no data) or `nonPD` (the arc's covariance is not positive definite — a mis-scaled `Cep` produces exactly this). A skipped arc keeps its previous factor; when more than `Noise.SkipThreshold` of a stream's arcs are skipped, the pass is refused rather than estimated on a fraction of the data.

7.5 Outliers

Gross outliers belong to preprocessing. Inside the adjustment there are three tools:

1. **Arc factors** down-weight a whole bad arc automatically (`ArcFactors = true`).
2. **Robust estimation** down-weights single epochs: `Robust = "igg3"` or `"bisquare"` together with `Estimate = "noise"` weights every epoch of every stream inside the joint estimator, standardised by the stream's own robust scale; a rejected epoch is removed from the arc exactly. The statistic is Baarda's w for correlated observations — what deleting the epoch removes from v'Pv — tested one epoch at a time, the largest first (data snooping), and the truncation of the epochs that are kept is accounted for lag by lag. So the level, the law and σ0 of a robust run equal those of a plain run on clean data, on a coloured stream too (within 0.25 % in variance, measured), and a small blunder on a strongly correlated stream is found: at a lag-1 correlation of 0.98 a blunder of 2 standard deviations stands out by about 14 in

w. Prefer a redescending function (`igg3`, `bisquare`) for gross blunders: Huber keeps part of their weight. The factors are in `R.noise.robust.factor`; a pass that rejects more than 45 % of the epochs warns that it has reached the breakdown point. A rejected set that flips back and forth between two passes is frozen at the union of the two (`R.noise.robust.frozen` names the pass).

3. **Diagnose, then decide.** Read the outlier tests in `R.report`, repair or remove the observations in `buildDay` and run again. Weights taken from a robust run can be made data: zero the rejected epochs' rows and mark them `excluded` in the day `buildDay` returns — the toolbox deletes such rows from a correlated law exactly, whole epochs or single axes (removing them leaves a gap that a `psd` or `arma` law, which counts lags in samples, takes for one step — `lsqvce:covModel:unevenGrid` says so); store the weights, never recompute them inside `buildDay`.

With `Estimate = "none"`, `Robust` runs a stand-alone iteratively re-weighted solve on the whitened innovations, with its own budget `RobustOpts`.

8 Priors and regularisation

8.1 The idea

A prior says "these parameters are observed to be x_0, with standard deviation σ". It is written as pseudo-observations `L·x = x₀ + v` in your own columns, and the toolbox treats it as data: it enters the normal equations, it has a redundancy (how much of it the data overrule), and its σ can be estimated like the level of any other stream. Regularisation is therefore never an option on the run: it is a declared part of the problem. The fields are listed in the API reference (*Priors*).

8.2 Typical priors

A datum for a free network. Minimum constraints on the null space N of the design (`L = Nᵀ`, `x0 = 0`) fix the datum without distorting the solution. Declare the datum as a group of its own:

```
data.priors(1) = struct('L', Nnull.', 'x0', 0, 'sigma', 1, 'name', 'datum');
```

The solution and its covariance are the minimum-constraint ones for any σ, also when the datum is declared as several groups (for instance translation and rotation separately). The datum rows are constraints, not observations: the report does not count them as uncontrolled. Without a datum a rank-deficient system is refused by name; the toolbox never guesses one.

A Kaula rule on a gravity field. One group per rule, σ estimated:

```
data.priors(1) = struct('group', '^[CS]', 'x0', 0, 'sigma', 1e-10, 'estimate', "scale");
```

`group` is a regular expression on the name column of `commonParams`; the toolbox builds one row per matched coefficient. Relative weights inside a group (degree-dependent Kaula factors) go into an explicit `L` instead: divide each row by its own relative σ.

Calibration parameters of an instrument. Priors on local parameters sit on the system:

```
data.systems(k).priors(1) = struct('group', '^accScale', 'x0', 1, 'sigma', 0.02, ...
                                   'estimate', "scale");
```

The accelerometer scales of both satellites over a month form one group with many rows — exactly what makes their σ estimable. A group with one row cannot estimate its own σ.

8.3 Writing `L` yourself

`lsqvce.layout(data)` announces which column of A and B holds which template row on which day and arc, so an operator can be built by name:

```
lay = lsqvce.layout(data);
L = zeros(2, lay.nc);
L(1, lay.colCommon(5, [])) = 1;            % template row 5 of commonParams
L(2, lay.colCommon(6, [])) = 1;
```

`examples/demo_priors.m` builds priors both ways.

8.4 Reading the result

`R.noise.priors` and `R.noise.priorHistory` report the σ of every common group per pass; a local group appears as the stream `prior_<name>` of its system in `R.noise.model`. $\lambda = \sigma_0^2/\sigma^2$ is the regularisation the data support. `R.report` counts the redundancy of the prior rows like that of any observation.

A group with `estimate = "scale"` is estimated only under `Estimate = "noise"` on the streaming kind. In any other run it keeps its declared σ, and `fit` says so once (`lsqvce:fit:priorScaleHeld`).

8.5 Activating a prior later

`from = 3` activates a group in the third pass of an iterative run: the first passes then see the data alone. A single solve skips such a group with a warning.

9 Reading the results and the report

`lsqvce.fit` returns a `lsqvce.Results` object. This chapter covers how to read it: first the object itself, then the five-level report that judges it, then the per-stream evidence of an `Estimate = "noise"` run, which the report does not print. For each adverse finding it says what to do. The full field lists are in the API reference, and the derivations of every statistic are in the theory.

```
R = lsqvce.fit(data, opts);
R.report                       % the verdict, levels 0..4
V = R.report(Verbose=false);   % the same verdict, silent, as a struct
```

9.1 The `Results` object

9.1.1 Properties

All properties are read-only.

property	what it holds
`R.x`	the common-parameter estimate, `nc x 1` (the eliminated local parameters are not in it; `R.toTable` and `R.residuals` give them back)
`R.sigma0`	the a-posteriori standard deviation of unit weight
`R.sol`	the solver's raw output: `.x`, `.sigma0`, `.Cxx` (the covariance, **already scaled by `sigma0^2`**), `.vtv`, ...
`R.diag`	the per-observation diagnostics the report reads (next sections)
`R.history`	the pass record of an iterative run; empty for a single solve
`R.noise`	the estimated stochastic model (`Estimate = "noise"`), or the robust record
`R.converged`, `R.stopReason`	whether the loop settled, and why it stopped

property	what it holds
`R.meta`	run provenance: `.toolboxVersion`, `.matlabRelease`, `.createdUTC`, `.seeds`, `.options` (as resolved), `.warnings`
`R.acc`, `R.source`, `R.batchFcn`, `R.nBatch`, `R.systems`	the accumulator, the diagnosed full-parameter solution and its replayable batch source (for expert use)

`R.stopReason` takes one of five values:

value	meaning
`"single"`	one solve, nothing iterated (`Estimate = "none"` without `Robust` or `x0`)
`"converged"`	every stop criterion was met
`"maxPasses"`	the pass budget ran out. The last pass is returned with its covariance and one warning: this is a result, not an error
`"userStop"`	your `Options.OnPass` hook returned `false`
`"diverged"`	Gauss–Newton found no downhill step; more passes cannot help

9.1.2 Methods

method	answers
`R.report(...)`	Is the adjustment acceptable? Prints the five levels and returns the verdict
`R.coefficients(Alpha=0.05)`	the estimates as a table: `index`, `estimate`, `se`, `lower`, `upper` (normal bounds, `se = sqrt(diag(Cxx))`)
`R.toTable(data, Estimate=9, Std=10)`	your parameter tables with the estimate and its standard deviation written into the named columns, one row per instantiated parameter (commons and locals)
`R.reliability()`	one row per observation: `obs`, `v`, `r`, `w`, `mdb`, `quadrant`
`R.snoop(...)`	iterative data snooping on this fit; `R` itself is not changed
`R.systematic(...)`	runs, Ljung–Box and binned-mean tests for unmodelled signal in the residuals
`R.compare(Rred)`	nested-model F test: does the full model `R` earn its extra parameters over `Rred`? Both fits must share one fixed weighting. After `Estimate = "noise"` each model's v'Pv is normalised to its own redundancy, so F would be 1 whatever the data, and such fits are refused. Declare the noise model the full fit estimated, with `estimate = "none"`, refit both, and compare those.
`R.ellipse(i, j, ...)`	confidence ellipse of the parameter pair `(i, j)`
`R.residuals(...)`	the collected residuals per system and stream (re-reads the arcs)

method	answers
`R.contribution(...)`	the information share of every stream in the common parameters
`R.summary()`	one row per **stream**: what each ended up as (iterative runs)
`R.convergence(...)`	one row per **pass**: when the loop settled (iterative runs)
`R.plot(kind, ...)`	`"residuals"`, `"contribution"`, `"psd"`, `"convergence"`. Returns one figure handle and exports nothing
`R.redundancy()`	`sum(r_i) = n − u`
`disp(R)`	a one-screen summary

Some methods cannot run on every result:

- `summary`, `convergence`, `plot("psd")` and `plot("convergence")` need a variance-component history. On a single solve they raise `lsqvce:Results:noHistory`.
- `residuals`, `contribution`, `toTable`, `plot("residuals")` and `plot("contribution")` need the streaming accumulator. On the two in-memory kinds they raise `lsqvce:Results:noAccumulator`; read `R.diag.v` there instead.
- `reliability` and `systematic` need every residual (`Diagnostics.Store = "all"`, the default). Under `"flagged"` or `"none"` they refuse (`lsqvce:Results:noPerObservation`, `lsqvce:systematic:noResiduals`). The report still runs, because it reads only aggregates and the flagged list.

Standard deviations. `R.sol.Cxx`, `R.coefficients` and `R.toTable` all carry the a-posteriori scaling. An arc's standard deviation in an `Estimate = "noise"` run has three factors: declared `sigma` × `sqrt(arc factor)` × `R.sigma0`. The chapter *Choosing the stochastic model* has the details.

9.2 How the report is built

`R.report` is `lsqvce.report(R.diag)`. It also passes along the run provenance and the loop state, so the printed report gets one provenance line at Level 0 and one loop-settled line at Level 1 (described below). A bare `lsqvce.report(R.diag)` prints neither line.

name-value	default	effect
`Verbose`	`true`	print the report; `false` only returns `V`
`Width`	78	line width, ≥ 40

The report always evaluates and prints all five levels. There is no option to print only one level; to filter, use the `level` field of the findings. The text goes through the toolbox's message log, like every other status line.

The returned struct:

```
V.verdict       % 'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
V.message       % the sentence printed under the verdict
V.findings      % cell of struct(level, severity, code, msg)
V.nError, V.nWarn, V.nNote, V.integrityOK
```

Each finding has a severity, `ERROR`, `WARN` or `NOTE`, and a stable **code**. A pipeline should branch on the codes, not on the wording:

```
codes = cellfun(@(f) f.code, V.findings, 'UniformOutput', false);
if any(ismember(codes, {'SNOOP_CONTAMINATED', 'SNOOP_REFUSE', 'SNOOP_BREAKDOWN'}))
```

```
    R2 = lsqvce.fit(data, opts, Robust="igg3");
end
```

9.2.1 The tests behind it: `Options.Diagnostics`

The diagnostics are computed by `fit` itself, in a second sweep over the data. Their settings belong to the `Diagnostics` option group:

field	default	meaning
`Alpha`	`1e-3`	significance of each per-observation test
`Bonferroni`	`true`	divide `Alpha` by `n`: the family-wise rate over all `n` tests
`Beta`	`0.2`	type-II error for the MDB (power 0.8)
`RMin`	`0.01`	redundancy number at or below which an observation is **uncontrolled**: its `w` is set to 0 and its MDB to `Inf`
`RWeak`	`0.1`	redundancy number below which an observation is **weakly controlled**
`Test`	`"auto"`	`"pope"` (studentised by the estimated `s0`), `"baarda"` (by the a-priori sigma0), `"mad"` (by a robust MAD scale); `"auto"` selects `"pope"`
`RobustCheck`	`true`	also form the MAD scale and the contamination gauge
`Store`	`"all"`	keep every per-observation vector, or only the `"flagged"` ones, or `"none"`
`NWorst`	`1000`	how many largest-`|w|` observations are retained (`R.diag.worstW`, `.worstIdx`)

```
opts = lsqvce.Options(Diagnostics=struct('Test', "mad"));   % fields merge into the defaults
```

What the numbers refer to.

- **Observations.** An observation index is a row in **sweep order**: the arcs in the order they were accumulated, then the rows of a declared prior. Prior rows are counted in `n` and in the redundancy.
- **Units.** On the streaming kind every row is whitened by its declared law before it is tested. `v`, `w`, `r_i` and the MDB therefore belong to whitened rows, and an MDB is in units of that row's own sigma. For correlated data only sums such as `sum(r_i)` and `vPv` are the same in both frames.
- **Parameters.** A parameter index in the report refers to the **full** parameter vector: the commons first (indices `1..numel(R.x)`), then each system's local parameters in block order. The `u` in the header counts both.
- **The a-priori sigma0.** For whitened rows the a-priori sigma0 is 1. The report's `s0` is the a-posteriori value recomputed from the residual sweep and should match `R.sigma0`; Level 0 checks this.

9.3 The verdict

The verdict is printed last, between two rules:

```
VERDICT: <verdict>        (<e> error, <w> warning, <n> note)
<message>
```

verdict	rule	what it means	what to do
`INVALID`	a Level-0 integrity check failed	The arithmetic or the setup is broken (for example the second pass did not replay the batches the solve saw). Levels 1–4 are **not evaluated**.	Fix the setup. Do not read or publish any number from this run.
`SUSPECT`	integrity holds, at least one `ERROR`	The adjustment computed, but at least one finding is serious enough that the result should not be published as it stands.	Work through the `[ERROR]` items, top level first.
`PASS WITH WARNINGS`	no `ERROR`, at least one `WARN`	Nothing disqualifies the result, but a reviewer will ask about the warnings.	Address each `[WARN ]` item or be ready to defend it.
`PASS`	no `ERROR`, no `WARN`	Every check passed.	Still read the `REL_EXTERNAL` line at Level 3. It is a `NOTE` whenever the reliability is normal for a network of this size, even when its absolute value is above one sigma.

The levels are ordered so that a failure at one level makes the levels below it unreliable. Read the report from the top, and fix the highest failing level first.

9.4 Level 0 — INTEGRITY (does the arithmetic hold?)

These checks test the computation, not the data.

code	severity	condition	action
`PROVENANCE`	NOTE	always (through `R.report`)	Prints the toolbox version, the MATLAB release and the creation time.

code	severity	condition	action
SUM_R	NOTE / **ERROR**	sum(r_i) against n − u; ERROR if the relative error > 1e-6	The second pass does not replay the design or the weights the solve used. Make buildDay / batchFcn deterministic: the same rows in the same order on every call.
VPV_SIGN	**ERROR**	vPv negative or non-finite	A numerical failure. Check the declared covariances for non-SPD blocks.
VPV_MISMATCH	WARN / **ERROR**	the accumulator's vPv and the recomputed sum p v² differ by > 1e-6 (WARN) or > 1e-2 (ERROR, invalid)	The recomputed value is the one the report uses. A large gap means cancellation in the normal-equation path; re-run with Backend = "qr".
NO_REDUNDANCY	**ERROR**	r = n − u = 0	Nothing is testable. Add observations or remove parameters.
S0_FALLBACK	WARN	the estimated variance factor was unusable	The w values were studentised with the a-priori sigma0. Read them as Baarda-type, not Pope-type.

9.5 Level 1 — GLOBAL (is the model consistent?)

Level 1 checks whether the functional and stochastic model are consistent overall.

Loop state and robust fits. If the run iterated (any stopReason other than "single"), one of these lines comes first:

code	severity	meaning	action
NOISE_SETTLED	NOTE	the loop met its stop rule; the line names the loop (the stochastic model, the robust re-weighting or the Gauss–Newton iteration) and its number of passes	—

code	severity	meaning	action
NOISE_UNSETTLED	WARN	the loop stopped on "maxPasses" or "userStop" before it settled, or a Gauss-Newton fit stopped on "diverged" (no downhill step). Every number below rests on weights or a point that were still moving	Treat the result as provisional. The noise loop and Gauss–Newton: raise MaxPasses or loosen Tolerance; the robust re-weighting: raise RobustOpts.MaxIter or loosen RobustOpts.Tol. For the noise loop R.convergence() shows what was still changing (see “When the loop did not settle” below); a robust or Gauss–Newton run lists its passes in R.history. For "diverged" more passes cannot help: check the start point and the Jacobian.
ROBUST_FIT	NOTE	the run used a robust influence (Robust = "huber", "igg3" or "bisquare")	sigma0 and the covariance belong to the **reweighted** problem. They are not an M-estimator’s scale or sandwich covariance, so the global test and the parameter bounds are approximate in both directions.

The global test. Two lines always appear:

- GLOBAL_STAT prints s0^2/sigma0^2 and T = vPv/sigma0^2, together with the chi-square acceptance band for r degrees of freedom. The band is fixed at a two-sided **α = 1 %** (quantiles 0.005 and 0.995, divided by r); it does not use Diagnostics.Alpha.
- GLOBAL_CI gives sigma0_hat with its **95 % confidence interval** and the smallest relative deviation this redundancy can tell apart from sampling noise. **Read the interval, not the point estimate.**

What follows depends on whether the ratio falls inside the band:

code	severity	condition	meaning and action
GLOBAL_PASS	NOTE	ratio inside the band	The model is consistent.

code	severity	condition	meaning and action
`GLOBAL_SHARP`	NOTE	ratio outside a band narrower than ±1 %	At this redundancy the test rejects any imperfection, however harmless. Judge the **ratio** and the confidence interval, not the rejection itself.
`GLOBAL_HIGH`	WARN	ratio above the band	The residuals are larger than the weights promised. There are three causes, and the global test cannot tell them apart: **(a) outliers**, see Level 2. **(b) Unmodelled systematics**: run `R.systematic(Covariates=...)` and plot `w` against time, elevation, orbit position and similar covariates; neither this test nor data snooping can see a smooth bias. **(c) Over-optimistic weights**: if the residuals are clean and structureless, estimate the weights with `Estimate = "noise"`, or correct the declared `sigma`. Do not rescale `Cxx` by `s0²` before you have ruled out (a) and (b).

code	severity	condition	meaning and action
GLOBAL_LOW	WARN	ratio below the band	The residuals are smaller than predicted. Either the weights are pessimistic (the declared sigma is too large; correct it or use Estimate = "noise"), or the model is **over-parameterised** and absorbs the noise. Test whether each parameter group is justified with R.compare(Rred), and look at the correlations at Level 4.

9.6 Level 2 — LOCALISE (where is it broken?)

Level 2 locates individual bad observations by data snooping: one test per observation on its normalised residual

w_i = (P v)_i / (scale · sqrt((P Qvv P)_ii)).

The scale is the estimated s0 (Pope), the a-priori sigma0 (Baarda) or the robust MAD scale. With Bonferroni = true the level of each test is Alpha / n. The critical value follows the scale: Pope's tau, the normal distribution, and for the MAD a Student t with r/2 degrees of freedom (calibrated to the nominal level; Pope's tau there gave up to 60 times the nominal false alarms on small networks). The price of the valid level is power on short networks under heavy contamination: with 50 observations and 10 % blunders of 25 sigma, four or more of five are found in about 60 % of cases (snooping with Test="mad" does not change that); from about 200 observations all of them.

9.6.1 The contamination cross-check

Pope's statistic divides by s0, and s0 is estimated from the same data that contain the blunders. A group of blunders therefore inflates the scale it is tested against, and the blunders mask each other. To catch this, the report compares s0 with a robust MAD scale of the raw standardised residuals and estimates the contaminated fraction fhat: the share of residuals beyond 2.5 robust sigmas. On clean data fhat sits near P(|Z| > 2.5) = 0.0124. As a backstop it also prints s0/sigma0, which contamination cannot suppress but which is only as good as your declared sigma0.

The null band is measured, for every n (the scored residuals): on clean data the count n·fhat is negative binomial with mean n · 0.0124 · (1 + 27/n) (a small sample's MAD is biased low) and variance 1.69 times that mean. D.fhatTol is its 99.75 % point (false alarms ≤ 0.25 %), and fhat counts as *collapsed* only where it is implausibly low for clean data (P(K ≤ n·fhat) ≤ 1 %). Below n ≈ 450 even fhat = 0 is common on clean data (59 % at n = 30), so there a captured robust scale cannot be told from a sigma0 declared too small (D.collapseDetectable = false).

The cross-check appears whenever RobustCheck = true, whatever the Bonferroni setting. Exactly one of these lines is printed:

code	severity	condition	action
SNOOP_SCALECHECK	NOTE	clean: fhat within its null band D.fhatTol	s0 and the robust scale agree, so the flag list can be believed. (This line is omitted under Test = "mad".)
SNOOP_INDETERMINATE	WARN	fhat within its null band, s0/sigma0 > max(3, 2·(1 + 3/sqrt(2r))), and a collapse undetectable at this n	Either sigma0 is declared too small (the common case: then the flag list can be believed) or more than half the data are bad and the robust scale is captured (then it cannot). Check sigma0 first; if it is trustworthy, fix the functional model before rejecting anything. R.snoop runs here with a warning (lsqvce:snoop:indeterminate), its rejections capped at MaxRejectFrac.
SNOOP_MODERATE	WARN	fhat above the null band, below 0.10	Pope's test is starting to lose power. Re-diagnose with Test = "mad", and reject one observation at a time (R.snoop(Test="mad")).
SNOOP_CONTAMINATED	**ERROR**	0.10 ≤ fhat < 0.25	Pope's test is **blind**: an empty flag list from a Pope run is not evidence of clean data, and iterating does not help. Use Test = "mad", or refit with Robust = "huber" ("igg3" / "bisquare" for gross blunders).

code	severity	condition	action
SNOOP_REFUSE	**ERROR**	fhat ≥ 0.25	So large a fraction of "outliers" points to a model error, not to outliers. Rejecting them would produce a confidently wrong result, and the flag list is incomplete anyway. Run R.systematic() and fix the functional model before rejecting anything.
SNOOP_BREAKDOWN	**ERROR**	fhat collapsed below its null band while s0/sigma0 > max(3, 2·(1 + 3/sqrt(2r)))	The robust scale has adopted the blunders as its reference (more than about half the data are bad). **Reject nothing** and trust no flag list in this report. Either the declared sigma or the functional model is badly wrong; fix that first.
SNOOP_NOCHECK	NOTE	no robust scale could be formed (too few usable residuals, or RobustCheck = false)	Pope's test is used unchecked. Above about 5 % contamination it loses power without warning.

In addition:

- SNOOP_BACKSTOP (NOTE) appears whenever s0/sigma0 exceeds its clean-data tolerance 1 + 3/sqrt(2r). It is not printed in the breakdown and indeterminate cases, which already report the backstop. If your sigma0 is trustworthy, the residuals really are that much larger than promised.
- SNOOP_MAD (NOTE) states that the robust scale replaced s0 (Test = "mad").

9.6.2 The snooping result

- SNOOP_SETUP (NOTE) prints the test, Alpha, the corrected level, n, the critical value |w| > k and the number of false alarms expected by chance (Alpha under Bonferroni).
- SNOOP_NOBONF (WARN) replaces it when Bonferroni = false. It prints the uncorrected level and roughly how many good observations will be flagged by chance. Set Bonferroni = true.
- SNOOP_VACUOUS (**ERROR**, Pope only) appears when the critical value comes so close to the upper bound sqrt(r) of Pope's statistic that a residual must carry at least 90 % of vPv to be flagged (k^2 / r >= 0.9, D.vacuityShare). Only a blunder that dominates the whole adjustment can then be found, and an empty list says nothing. Accept a weaker Alpha or a weaker family-wise correction, or accept that this network cannot detect its own blunders.
- SNOOP_CLEAN (NOTE) means nothing exceeds the critical value. It prints the largest |w| and its observation index.

- SNOOP_FLAGS (WARN) gives the number flagged, the number expected by chance, and so about how many are real. SNOOP_TABLE (NOTE) lists up to 15 of them, worst first, with columns obs, w, r_i, v, MDB. Read w and r_i **together**.
- SNOOP_LOWR (WARN): some flagged observations have r_i < RWeak. A large residual on a weakly controlled observation may be error absorbed from a neighbour. Rejecting it is close to a coin flip; prefer to re-measure.
- SNOOP_ITER (NOTE) prescribes the remedy. Reject **one** observation at a time (the largest |w|), re-solve and test again, because blunders mask each other and a single pass misses some of them:

```
[D2, REJ, sol2] = R.snoop();                % Pope, Bonferroni, one at a time
[D2, REJ, sol2] = R.snoop(Test="mad");      % under contamination
REJ.idx      % rejected, in order;  REJ.refused  % tripped but r_i < RMinReject (0.10)
```

R.snoop takes Test ("pope" or "mad"), Alpha and Bonferroni name-values; left out, they are R.diag's, so Options.Diagnostics reaches them. It caps the number of rejections at max(10, 0.01·n) and stops at a fraction of 0.25 (MaxRejectFrac). It refuses to run at SNOOP_BREAKDOWN and SNOOP_REFUSE (Force = true proceeds with a warning), warns at SNOOP_INDETERMINATE (and at SNOOP_CONTAMINATED under Pope's test), and tests only what R.diag tests (r_i > RMin). It refuses observations with r_i < RMinReject, re-solves cleanly over the survivors at the end, and leaves R unchanged. A prior's rows, common or local, are never rejected, and the re-solve keeps the prior as a prior (a datum stays a datum); the re-diagnosis keeps R.diag's thresholds and storage (Store, NWorst, RobustCheck). The returned D2 counts in the original observation numbers (D2.flagIdx, D2.rejectedIdx, ...); its per-observation vectors cover the survivors, and D2.obsIdx names their original numbers. It cannot reject single rows of a correlated batch (lsqvce:snoop:correlatedBatch): there, remove the epoch in buildDay or use Robust.

9.7 Level 3 — RELIABILITY (what can you not check?)

Level 3 shows which observations the tests could not check at all.

- REL_HEAD (NOTE) prints the smallest redundancy number and its observation, the mean redundancy r/n, and the MDB non-centrality delta0. delta0 is computed at the **Bonferroni-corrected** level and Beta: 4.13 at a level of 1e-3 with Beta = 0.2, and larger as n grows. It is the normal (Baarda, known-sigma) value. Under Test = "mad" the threshold is Student t with r/2 degrees of freedom, so the bias that can hide from that test is larger (about +50 % at n = 30, +25 % at n = 50, +6 % at n = 200); the line then says that the MDB, the envelope and the external reliability are the Baarda values.
- REL_QUADRANTS (NOTE) counts the observations in each cell of the (|w|, r) table. |w| is high when it exceeds the critical value; r is low when it is below RWeak.

quadrant	|w|	r	meaning
clean outlier	high	high	reject with confidence
AMBIGUOUS	high	low	could be a neighbour's error; re-measure
verified	low	high	tested, and passed
UNTESTED	low	low	**not** "fine": the test had no power

A residual shows only the fraction r_i of an error in its observation. A small |w| on a low-r observation is therefore no evidence of health. R.reliability() gives the same quadrant per observation; the rows of a datum prior carry the label "datum" and are not counted in any quadrant.

code	severity	condition	action
`REL_SILENT`	WARN	observations in the UNTESTED quadrant	These observations did not pass the test; the test could not see them. Strengthen the geometry (more or better-placed observations), or validate them externally.
`REL_AMBIGUOUS`	WARN	flagged observations with `r_i < RWeak`	Re-measure rather than reject.
`REL_ENVELOPE`	NOTE	always, when MDBs are finite	The hidden-blunder envelope: the median, 90th and 99th percentile and the worst MDB, in units of each observation's sigma. "Half of my observations could hide an undetected error larger than the median." The percentiles come from a fixed-seed reservoir of 5000 samples.
`REL_MDBINF`	**ERROR**	observations with infinite MDB	No error in these observations, however large, can be detected. They drop out of the percentiles above, so the envelope looks better than it is.
`REL_UNCONTROLLED`	**ERROR**	observations with `r_i ≤ RMin`	Their residual is forced to zero, so a blunder in them passes into the parameters unseen. The first indices are listed. Add observations that constrain the same parameters, or remove the parameter that only those observations determine.
`REL_WEAK`	WARN	`RMin < r_i < RWeak`	A blunder in these is only partly visible. Data snooping has little power here.
`REL_GOOD`	NOTE	none weak, none uncontrolled	Snooping has power everywhere.

External reliability (`REL_EXTERNAL`) answers one question: how far could the worst undetectable blunder (one MDB in the worst observation) move a parameter, in units of that parameter's own sigma? The report names the observation and the parameter. Under a healthy geometry this number shrinks like `1/sqrt(n)`, so it is judged against the value expected for a network of this size, `4.0 · delta0 / sqrt(n)`:

ratio to expected	severity	action
≤ 1.8	NOTE	Normal for this size.
1.8 – 3	WARN	The geometry is weaker than it should be.
> 3	**ERROR**	One observation carries far more leverage than the rest. Find it through the minimum `r_i` and the named observation, and do not trust the formal sigmas.

If the absolute value exceeds one sigma, the line adds that an undetectable blunder could move the parameter by more than its stated precision. When that happens at a normal ratio, the remedy is more observations, not a different estimator.

9.8 Level 4 — PARAMETERS (what do you actually know?)

Level 4 checks conditioning and correlations, and lists the parameter precisions.

- `PAR_COND` (NOTE) prints `cond(R)` and `cond(N)`. On the `fit` path these are **lower bounds** formed from the blocks of the cofactor (no SVD), and they are marked `[LOWER BOUND]`. The true conditioning can be worse than printed, never better, so the thresholds below are optimistic.
- `PAR_ILLCOND`: WARN when `cond(N) > 1e10`, with roughly `log10(cond(N))` of 16 digits lost. **ERROR** when `cond(N) > 1e14`: the printed sigmas are meaningless. Rescale the parameters so that the columns of the design have comparable norms, and use `Backend = "qr"`, which avoids squaring the condition number. A rank defect (a free network) needs a datum, which you declare as a prior (chapter *Priors and regularisation*).
- `PAR_CORR` reports the strongest correlation and its parameter pair. It is a NOTE at most 0.95, a WARN above 0.95 (nearly inseparable: use `R.ellipse(i, j)` rather than the individual sigmas), and an **ERROR** above 0.999. In the ERROR case only a combination of the two parameters is determined. Fix one of them, reparameterise to estimate the combination, or add observations that separate them.
- `PAR_TABLE` (NOTE) lists the first 30 parameters of the full vector: `x`, the a-priori sigma (`sigma0 · sqrt(Qxx_jj)`) and the scaled sigma (`s0 · sqrt(Qxx_jj)`). The scaled column is what `R.coefficients` reports for the commons.
- `PAR_SCALE` (WARN) appears when `|s0²/sigma0² − 1| > 0.1`, so that the two columns differ. Which column is honest depends on **why** the variance factor deviates. If the weights were wrong, the scaled column is right. If there are outliers or systematics, **neither** is right. Resolve Levels 1 and 2 before quoting either.

9.9 Beyond the report: the stochastic model

The report judges one solve under the final weights. Whether those weights describe the noise, stream by stream, is a separate question, and the report does not answer it. For an `Estimate = "noise"` run, read it from `R.history`, `R.summary()` and `R.convergence()`.

9.9.1 Whiteness per stream

On every pass the loop tests the law-whitened residuals of each stream with a Ljung–Box test. The test pools over the stream's arcs, uses **K = 20 lags** and calls a stream white when **p > 0.05**.

```
T = R.summary();              % stream | sigma | arOrder | white | pValue  (final pass)
T(~T.white, :)                % the streams that never whitened
R.history.streams             % stream keys 'satKey|obsType', the column order of
R.history.white(end, :)       %   these per-pass matrices (nPass x nStream)
R.history.whitePval(end, :)
```

In `summary()`, `sigma` is the stream's level factor `sqrt(mean arc factor)`. It is near 1 when the declared sigma was right, and it tells you by how much the declared sigma was off. `arOrder` is `NaN`, because the joint estimator fits a spectrum, not an AR order.

A stream that is not white has residual correlation that its law does not describe. Possible causes and remedies, roughly in order:

1. **The law was not estimated for it.** If the stream's covariance spec has `estimate = "none"` or `"scale"`, the declared law is used unchanged (its `R.noise.model{k}.C` is then empty). Set `estimate = "law"` for that stream.
2. **It shares a law with streams that behave differently.** Use a finer `Pool` (`"obsType"`, or `"stream"` for one law per stream).
3. **The spectral grid is too coarse** for its correlation: raise `Noise.Grid`.
4. **The functional model is incomplete.** An unmodelled signal (a drift, a once-per-revolution term) appears as coloured noise. Run `R.systematic(Covariates=struct('time', t, ...))`: the binned-mean test points to the covariate. A correlation that lives only at a long lag (an echo, a repeat pattern) is invisible at K = 20; test it with `R.systematic(Lags=L)`.

`R.plot("psd")` draws the estimated spectral shape of each pool group, and `R.plot("residuals", Stream="...")` draws a stream's whitened residuals against time.

9.9.2 When the loop did not settle

The loop stops when two conditions hold. Every arc factor changed by at most `Tolerance`. Every spectral bin j changed by at most `Tolerance·max(1, sqrt(200/r_j))`, where `r_j` is the bin's redundancy and `sqrt(2/r_j)` the relative standard error of its estimate. A bin with a redundancy of 200 or more is held to `Tolerance`. A bin with less, such as the zero-frequency bin that per-arc parameters absorb, may move further, in proportion to its own standard error: at the default `Tolerance` 1e-2, by a tenth of it. A finer step would ask for more precision than its estimate has. A smaller `Tolerance` tightens every bin. (Before 6.26.0 every bin was held to `Tolerance`. On the fixtures of the development record that took 14–168 passes, while the solution was already within 0.1–0.2 of its standard error of the end point after 10.)

Each pass records `R.history.binRatio` (the largest change of a bin divided by its bound, so ≤ 1 means every bin settled), `R.history.levelDev`, `R.history.maxDev` (the largest `|α² − 1|`, a reading) and `R.history.xStep` (the largest change of the solution since the pass before, in units of its own standard error). `R.convergence()` gives one row per pass with `pass`, `maxSigmaOffset` (the largest `|sigma − 1|` of the per-stream level factors: a reading, not the stop rule), `nWhite`, `minWhitePval`, `converged`, `maxDev`, `levelDev`, `binRatio` and `xStep`. `R.convergence(Plot=true)` and `R.plot("convergence")` draw the same record.

With `stopReason = "maxPasses"`, the warning names the bins that still move, with their redundancy against the median of their group, and the solution's last step:

- If `xStep` is a few hundredths or less, the solution has settled to well within its errors, and the bins that still move are ones the data barely determine.
- If `binRatio` is still falling, the loop needs more passes: raise `MaxPasses`.

- If it oscillates, there is too little redundancy per component: coarsen `Pool` (`"obsType"` or `"all"`), use a smaller `Noise.Grid`, or smooth the spectrum (`Noise.Smooth`). A spectrum that keeps moving also points to too few arcs per frequency, or a start far from the data.
- If one stream stays non-white on every pass, see "Whiteness per stream" above.

9.9.3 Skipped arcs and the warnings inventory

An arc that cannot be used in a pass is **skipped and keeps its previous factor**. The reason is `'gap'` or `'nonPD'` (its covariance could not be factored). `R.history.skips` lists every skip as `{pass, arcKey, reason}`. A pass that skips more than `Noise.SkipThreshold` (0.2) of one stream's arcs is refused with `lsqvce:adjustVCEjoint:skipThreshold`. A few `nonPD` skips usually mean a `Cep` or a law that is not positive definite on those arcs, so inspect that day's `buildDay` output.

Warnings are raised once per run. Muted or not, every warning identifier the run raised is listed in `R.meta.warnings` (first-fired order). Check it in batch jobs. The ones that qualify a result:

identifier	meaning
`lsqvce:adjustVCEjoint:notConverged`	the stochastic model did not settle (the same fact as `NOISE_UNSETTLED`)
`lsqvce:adjustVCEjoint:arcFactorsDiverging`	the spread of a stream's arc factors kept growing over the passes: they are not an estimate. Take the one-pass estimate (`MaxPasses = 1`) or set `arcFactors = false` on that stream and read its level
`lsqvce:robust:atBreakdown`	a robust pass zeroed more than 45 % of the epochs (rows); the data are not a few blunders on a good model
`lsqvce:adjustRobust:notConverged`, `lsqvce:adjustNonlinear:maxIter`	IRLS or Gauss–Newton exhausted its budget (`stopReason = "maxPasses"`)
`lsqvce:systematic:lbUnreliable`	`R.systematic`'s Ljung–Box is anti-conservative at this `u/n`; lead with the binned test

9.9.4 Robust runs

A robust run reports `ROBUST_FIT` at Level 1. Its weights come back in `R.noise.robust` when the robust pass runs inside the noise loop: `influence`, `scale` per stream, `factor` per epoch in sweep order, `keys`, `truncation` (the factor the hard σ0 is corrected by) and `frozen` (the pass a cycling rejected set was frozen at, 0 if none). For the stand-alone IRLS they are in `R.noise.influence` and `R.noise.scale`, with per-pass `scale`, `dx`, `nDown` and `nZero` in `R.history`. Down-weighted epochs are the robust counterpart of a flag list. Compare them with a Pope or MAD snooping run before you remove anything permanently.

9.10 Adverse findings at a glance

you see	first suspect	do
`INVALID` (`SUM_R`, `VPV_*`, `NO_REDUNDANCY`)	setup or arithmetic	Make the data source deterministic, use `Backend = "qr"`, add redundancy. Read nothing else.
`NOISE_UNSETTLED`, `stopReason = "maxPasses"`	budget or poorly separable components	`R.convergence()`, then raise `MaxPasses`, loosen `Tolerance` or coarsen `Pool` / `Noise.Grid`

you see	first suspect	do
`NOISE_UNSETTLED`, `stopReason = "diverged"`	Gauss-Newton found no downhill step	check the start point `x0` and the Jacobian against a finite difference
`GLOBAL_HIGH`, flags present	outliers	`R.snoop()`, one at a time
`GLOBAL_HIGH`, no flags	systematics or optimistic weights	`R.systematic(Covariates=...)`; if clean and structureless, `Estimate = "noise"` or correct `sigma`
`GLOBAL_LOW`	pessimistic weights or over-parameterisation	correct `sigma` / `Estimate = "noise"`; `R.compare(Rred)`; Level-4 correlations
`GLOBAL_HIGH/LOW` with `GLOBAL_SHARP`	very large `r`	judge the ratio and the confidence interval, not the rejection
`SNOOP_MODERATE` / `SNOOP_CONTAMINATED`	contamination masks Pope	`Diagnostics.Test = "mad"`, `R.snoop(Test="mad")`, or `Robust`
`SNOOP_REFUSE` / `SNOOP_BREAKDOWN`	model error, or wrong sigma0	reject nothing; fix the functional model or the declared sigma
`SNOOP_VACUOUS`	redundancy too small for the corrected level	weaker `Alpha`, or accept that blunders cannot be detected
`SNOOP_LOWR`, `REL_AMBIGUOUS`	weak geometry	re-measure, do not reject
`REL_UNCONTROLLED`, `REL_MDBINF`, `REL_SILENT`, `REL_WEAK`	untestable observations	strengthen the geometry; validate those observations externally
`REL_EXTERNAL` WARN/ERROR	one observation with excessive leverage	find it (minimum `r_i`, named observation); distrust the formal sigmas
`PAR_ILLCOND`	scaling or rank defect	rescale columns, `Backend = "qr"`, declare a datum prior
`PAR_CORR` ERROR	inseparable parameters	fix or reparameterise, or add separating observations
`PAR_SCALE`	variance factor ≠ 1	resolve Levels 1–2 before quoting either sigma column
a stream not white in `R.summary()`	stream's law or the functional model	`estimate = "law"`, finer `Pool` / `Noise.Grid`, `R.systematic`

9.11 A worked reading

Every report has the same layout. The numbers below are placeholders; `demo_lsqvce` runs six small adjustments, each with one failure planted (clean, outliers, a systematic drift, an uncontrolled observation, wrong weights, collinear parameters), and prints the verdict of each, so you can see the real output for each case.

```
==========================================================================
  LEAST-SQUARES ADJUSTMENT REPORT
  n = <n> observations,  u = <u> parameters,  r = n-u = <r>
  <b> batches,  vPv source: accumulator (v'v)
==========================================================================

--- LEVEL 0: INTEGRITY  (does the arithmetic hold?)
       run: lsqvce <version>, MATLAB R<release>, created <UTC time>
       sum(r_i) = <value> vs n-u = <r>  (rel. err <e>)  [identity holds]
```

```
--- LEVEL 1: GLOBAL     (is the model consistent?)
       The stochastic model settled in <k> pass(es).
       s0^2/sigma0^2 = <ratio>     T = vPv/sigma0^2 = <T>
       chi2 acceptance band (alpha=1%, r=<r>): [<lo>, <hi>]  width <w>
       sigma0_hat = <s0>,  95% CI [<a>, <b>]   (a priori sigma0 = 1.00000)
       ...
       Global test PASSES: functional and stochastic model are consistent.

--- LEVEL 2: LOCALISE   (where is it broken?)
       Scale cross-check: s0 = <s0>, robust scale = <m> (ratio <q>), ...
       Data snooping: POPE test, alpha = 1.0e-03 per test, Bonferroni-
       corrected to <alpha/n> for n = <n> simultaneous tests.
       Critical value |w| > <k>.   Expected false alarms: 0.00
       No observation exceeds the critical value. Largest |w| = <w> ...

--- LEVEL 3: RELIABILITY(what can you not check?)
       Redundancy numbers: min = <rmin> (observation <i>), mean = <r/n>
       MDB non-centrality delta0 = <d0>  (alpha_eff = <..>, beta = 0.20)
       <quadrant table>
       Hidden-blunder envelope -- ...  median <m>  90th pct <..>  ...
       Every observation has r_i >= 0.10: ...
       External reliability: the worst undetectable bias (the MDB of
       observation <i>) would shift parameter <j> by <e> of its own sigma.
       That is normal for a network of this size ...

--- LEVEL 4: PARAMETERS (what do you actually know?)
       cond(R) >= <..>     cond(N) >= <..>   [LOWER BOUND]
       ...
       Maximum parameter correlation is <c> (parameters <i> and <j>).
       <parameter table: par | x | sigma(a priori) | sigma(scaled)>

============================================================================
  VERDICT: PASS        (0 error, 0 warning, <n> note)
  All checks passed: ...
============================================================================
```

How to read it, in order:

1. **Level 0.** Check that `[identity holds]` is printed and that no `[ERROR]` appears. If one does, stop: the verdict is `INVALID`.
2. **Level 1, loop line.** If the run iterated, check that the stochastic model settled. If not, every later line is provisional.
3. **Level 1, the ratio.** Read the confidence interval on sigma0, not the pass/fail. A ratio well above 1 sends you to Level 2 (outliers) and to `R.systematic`.
4. **Level 2.** Read the scale cross-check before the flag list: if the check reports contamination, the list is not to be believed. With flags, read each `w` next to its `r_i`, and reject with `R.snoop`, one observation at a time.
5. **Level 3.** Check that the quadrant table has no UNTESTED or AMBIGUOUS counts and that no uncontrolled or infinite-MDB observations are reported. Check that the external reliability is normal for this `n`.
6. **Level 4.** Check that the conditioning is far from 1e10 and that no correlation is above 0.95. If `PAR_SCALE` fires, decide which sigma column applies only after Levels 1 and 2 are resolved.
7. **Outside the report.** For a `noise` run, check `R.summary()` for non-white streams, `R.history.skips`, and `R.meta.warnings`.

Then quote the result: `R.coefficients()` for the commons, `R.toTable(data)` for every parameter

in your own table layout, and `R.meta` for the provenance.

10 Watching a run

10.1 During the run: `OnPass`

`Options.OnPass` is called after every pass of every iterative run (the noise loop, IRLS, Gauss–Newton) with the pass state, the live accumulator, your declaration and your `processingOptions`. Plot, `drawnow`, save — or return `false` to stop after the pass:

```
function go = myInspect(ctx)
    res = lsqvce.engine.residuals(ctx.acc, struct('x', ctx.x, 'sigma0', ctx.sigma0, ...
              's0sq', ctx.s0sq, 'Cxx', ctx.Cxx), 'Verbose', false);
    figure(1); plot(res.sys(1).obs(1).t, res.sys(1).obs(1).w, '.'); drawnow;
    fprintf('pass %d: largest change %.3g\n', ctx.pass, ctx.maxDev);
end

R = lsqvce.fit(data, Estimate="noise", OnPass=@myInspect);
```

The fields of `ctx` are listed in the API reference (`lsqvce.Options`, *OnPass*). An error inside the hook stops the run and arrives as `lsqvce:callUser:threw` with your error as its cause. A stop by the hook is reported as `R.stopReason = "userStop"`, with the last pass returned in full.

`Verbose = true` prints one progress line per stage; the default is silent. `examples/demo_messaging.m` shows the output side by side.

10.2 After the run: one figure per question

```
R.plot("residuals")      % whitened residuals per stream
R.plot("psd")            % the estimated spectra per pool group
R.plot("convergence")    % the components pass by pass
R.plot("contribution")   % which stream determines which parameter
```

Each call returns one figure handle; nothing is exported by the toolbox. `R.stopReason` says why the loop ended (`"single"`, `"converged"`, `"maxPasses"`, `"userStop"`, `"diverged"`), and `R.meta.warnings` lists every warning identifier the run raised.

10.3 Reproducing a run

`R.meta` records the toolbox version, the MATLAB release, the creation time, every random seed the run resolved and the options as resolved. Saving `R` keeps all of them; a run with the same data, options and `Seed` reproduces the same numbers.

11 The examples

The folder `examples/` holds runnable demonstrations of the user path: build a `data` struct, choose `lsqvce.Options`, call `lsqvce.fit`, and read the `lsqvce.Results`. The folder `template/` holds starting points that you copy into your own project. Every example generates its own synthetic data from a seeded random stream, except `demo_gravfire`, which reads a private data bundle that is not shipped. All of them print their findings to the Command Window. A few of them also return their results so that you can inspect them afterwards.

`lsqvce_setup` puts `examples/` on the path. It does not add `template/`: copy those files into your project. `demo_gravfire` adds `template/` to the path by itself.

11.1 Which example for which question

Question	Example	Kind of data
I am new: what does a complete streaming run look like, and how do I use `prepareDay`?	`demo_tutorial`	streaming, one system, two streams
How do I wire my own data into `data` and read the verdict?	`demo_workflow` (script)	batched; streaming recipe
What do the three data kinds look like in one call each?	`demo_fit`	in-memory, batched, streaming
What does the report say when an adjustment is wrong in a specific way?	`demo_lsqvce` (script)	in-memory, one streaming case
How do I tell the toolbox what I know about each stream's noise?	`demo_noisePolicy`	streaming, one system
How do I regularise, or give a rank-deficient problem a datum?	`demo_priors`	streaming, one system
How are several systems with their own local parameters combined?	`demo_combine`	streaming, three systems
How do I declare system / sensor / day, with data gaps, and find a bad day?	`demo_sensors`	streaming, two systems
What does a full noise-estimation run on multi-axis streams look like?	`demo_grace` (+ `demo_grace_dataLoader`)	streaming, two systems
Does the noise estimation still work on a physically structured gravity design?	`demo_gravity` (+ `demo_gravity_dataLoader`, `demo_gravity_design`)	streaming, three systems
How do I map a GRAVFIRE preprocessing bundle onto `data`?	`demo_gravfire` + `template/gravfire_data.m`	streaming, real bundle
How do I build GRACE-FO range-rate observation equations from Level-1B files?	`template/kbr_variational.m`	streaming, real data
What does each verbosity level print?	`demo_messaging`	streaming
Should I use `Backend="normal"` or `Backend="qr"`?	`bench_qr_vs_normal`	batched

11.2 Suggested reading order

0. **`demo_tutorial`**: the Manual's chapter *Your first adjustment*, as a file.
1. **`demo_workflow`**, then **`demo_fit`**: the shape of `data`, the three kinds, `lsqvce.Options`, and how to read `R.diag` and `R.report()`.
2. **`demo_lsqvce`**: what the diagnostics say when something is wrong.
3. **`demo_noisePolicy`**: the per-stream noise policy (`estimate`, `arcFactors`).
4. **`demo_priors`**: `lsqvce.layout` and declared regularisation.

5. **demo_combine**, then **demo_sensors**: several systems, local parameters, data gaps, attribution by system, sensor and day.
6. **demo_grace**, then **demo_gravity**: complete noise-estimation runs on GRACE-like problems.
7. **demo_gravfire** / **gravfire_data** and **kbr_variational**: the templates for real data.
8. **demo_messaging** and **bench_qr_vs_normal**: what a run prints and which backend to use, on your own machine.

11.3 Examples, helpers and templates

Not every file in `examples/` is a demonstration. Three of them are **helper files** that the demos call:

Helper	Called by	Returns
`demo_grace_dataLoader`	`demo_grace`, `demo_messaging`	`[cfg, data]`: a GRACE-FO-shaped declaration with a random design
`demo_gravity_dataLoader`	`demo_gravity`, `demo_fit`	`[cfg, data]`: a gravity declaration with a physical design and planted truth
`demo_gravity_design`	`demo_gravity_dataLoader`	`[A, meta]`: spherical-harmonic potential partials along an orbit

Each helper also runs on its own and returns a ready `data` struct that you can pass to `lsqvce.fit`.

`demo_workflow` and `demo_lsqvce` are **scripts**, not functions. They begin with `clear`, so they wipe the base workspace. Run them in a fresh session, or section by section in the Editor. All other examples are functions.

11.4 How to run them

The call forms below are the ones the test suite uses to run every file in `examples/` once (with figures hidden). Each call is the cheapest form that still runs the whole example:

```
bench_qr_vs_normal('u', [10 20], 'n', 2e4, 'nk', 2000)
demo_combine()
demo_fit()
demo_grace('verbose', false)
demo_grace_dataLoader()
demo_gravfire(bundleFile)             % the suite writes a generated bundle to bundleFile
demo_gravity()
demo_gravity_dataLoader()
orbit = struct('u', linspace(0, 4*pi, 400).', 'lam0', linspace(0, 0.3, 400).', ...
               'incl', 89*pi/180, 'r', 1.07);
demo_gravity_design(4, orbit)
demo_lsqvce                           % script
demo_messaging()
demo_noisePolicy()
demo_priors()
demo_sensors()
demo_tutorial()
demo_workflow                         % script
```

`demo_gravfire` runs on a bundle the suite generates (`dev/tests/fixture_gravfire_bundle.m`, random numbers in the real structure), so `template/gravfire_data.m` runs with it. `demo_gravfire` with a path that does not exist prints what it would need and returns. `kbr_variational` needs Level-1B files and is not part of the run.

11.5 `demo_tutorial`: your first streaming run, with `prepareDay`

```
[R, s] = demo_tutorial();
[R, s] = demo_tutorial('verbose', true, 'plot', true);   % defaults: false, false
```

The file of the chapter *Your first adjustment*, line for line. One system watches a periodic signal with two instruments (`'gauge'`, white; `'radar'`, AR(1) but declared white) on two arcs a day for five days; every arc has an unknown offset. Its `prepareDay` computes the epochs and the signal basis of a day once, and both streams' `buildDay` read them from `procOpt.prepared`.

It runs `fit` twice — once under the declared noise, once with `Estimate = "noise"` — and prints the two estimates of the two common parameters against the planted truth. Both are close to the truth; the declared run's formal errors are about 1.6 times too small, because the radar's colour was missing. The noise run converges in four passes, $\sigma_0 = 1.000$, and the report's verdict is `PASS`.

`s` returns the truth, both runs' estimates, standard errors and σ_0, each stream's final level relative to the declared one, and the verdict. The test suite checks these claims (`test_fit`, `testDemoTutorialIsTheManualsTutorial`).

11.6 `demo_workflow`: build your own case

Purpose. This is the from-scratch recipe for wiring your own data into `data`, letting `lsqvce.fit` pick the engine from the shape of the struct, choosing what to estimate, and reading the result in the right order.

Run. `demo_workflow` is a script with no arguments and no outputs. It runs `clear; clc; close all`. The help text recommends running it section by section (Ctrl+Enter).

Outline and logic. The script is organised as Sections 0 to 5:

- *Section 0* lists the three kinds of `data`, which `lsqvce.fit` tells apart by the fields present:
  - in-memory: `A`, `l`, `p`;
  - batched: `batchFcn`, `numBatches`, `numUnknowns`, or `x0` for the nonlinear form;
  - streaming: `commonParams`, `days`, `systems`.

  It also states the batched callback contract `[A, l, p] = batchFcn(k)`, with `p` the weights `1./sigma.^2`. The callback is called twice per batch and must return identical data both times.
- *Section 1* synthesises 3000 observations with 8 unknowns and cuts them into 30 batches with a closure over precomputed batch edges. It sanity-checks `batchFcn(1)` with an `assert` and builds `data = struct('batchFcn', batchFcn, 'numUnknowns', u, 'numBatches', nb)`.
- *Section 2* is a decision tree in comments: which kind, and when to use `Estimate="noise"`. It then runs `R = lsqvce.fit(data)` with default options.
- *Section 2b* reads the verdict in a fixed order:
  1. integrity, `R.diag.sumRRelErr`;
  2. variance, `R.diag.varRatio`;
  3. blunders, `R.diag.nFlag`;
  4. reliability.

  The one-line verdict comes from `R.report('Verbose', false).verdict`.
- *Section 3* is a recipe only, with no runnable code. It shows the streaming declaration with `commonParams`, scoped `localParams` and one `covariance` field per observation type, and points to `demo_combine`, `demo_sensors` and `demo_grace`.

- *Section 4* plants a second instrument group that is three times noisier than claimed. It declares the two groups as two observation types of one streaming system with one day. The inline `buildDay` returns one arc per group, and the claimed per-row sigmas travel as `Cep`. Both covariance specs are `struct('type','diag','sigma',1,'estimate',"scale")`. The run uses `Options(Estimate="noise", Pool="stream", ArcFactors=false)`. The script prints the recovered sigma ratio from `Rv.history.sigma(end,:)` (the truth is 3) and `Rv.sigma0`.
- *Section 5* prints the condensed decision tree and the rule for reading a result.

11.7 `demo_fit`: the three data kinds, one call each

Purpose. This example shows every kind of `data` that `lsqvce.fit` accepts, end to end on the user tier: `lsqvce.fit`, `lsqvce.Options`, `lsqvce.Results` and `R.report`. It also shows robust estimation, a datum declared as a prior, Gauss–Newton through `x0`, and watching a run with `OnPass`.

Run.

```
out = demo_fit();                  % prints the verdicts
out = demo_fit(Verbose=false);     % silent
```

`out` has the fields `.matrix`, `.robust`, `.datum`, `.batch`, `.nonlinear`, `.gls` and `.vce`. Each of them is an `lsqvce.Results`. `out.truth` holds the planted values: `x = [1; -2; 0.5]`, `[a; b] = [0.7; -0.2]`, and `xGravity` of the gravity loader.

Logic.

1. **In-memory.** The data are 30 observations with `A = [1, randn, randn]`, seeded with 17, and the call is `data = struct('A',A,'l',l,'p',p)`. The same problem is then run with a 25-sigma gross error at row 7 and `Options(Robust="huber")`. The demo prints the IRLS weight factor of that row from `out.robust.history(end).factor(7)`.
2. **Free network.** The problem has height differences between four points and is rank-deficient by a common level. The datum is declared as `data.priors = struct('L', ones(1,4), 'x0', 0, 'sigma', 1)`. The demo prints `L*x`. It then removes `priors` and prints the error identifier with which `fit` refuses the problem.
3. **Batched, linear.** The same 30 rows are split into two batches with `struct('batchFcn',bf,'numUnknowns'`
4. **Batched, nonlinear.** The model is `l = exp(a t) + b`. The callback `bfn(k, x)` returns the Jacobian at `x`, and `data.x0 = [0.5; 0]` (the number of unknowns is `numel(x0)`). The options are `Options(MaxPasses=15, Tolerance=1e-9)`, and the demo prints `[a; b]`.
5. **Streaming.** `[cfg, data] = demo_gravity_dataLoader()` is solved once with defaults (`out.gls`). It is then solved with `Options(Estimate="noise", Pool="stream", ArcFactors=false, MaxPasses=2, OnPass=watch)`, where `watch` prints `ctx.pass`, `ctx.maxDev` and `ctx.sigma` after every pass. Two passes are not enough to converge, so the demo suppresses the warning `lsqvce:adjustVCEjoint:notConverged`. It prints `out.vce.history.sigma`.

Implementation. The local function `show` calls `R.report('Verbose', true)` and prints `R.sigma0`. According to the help block, the in-memory estimates land within 3 formal sigma, and the nonlinear form recovers `[a; b]` to 5e-3.

11.8 `demo_lsqvce`: six failure modes and their verdicts

Purpose. This example teaches you to read the diagnostics. Each of six scenarios isolates one failure, so that you see what `R.diag` and `R.report()` say when one specific thing is wrong, and what they say when nothing is.

Run. `demo_lsqvce` is a script with no arguments. It runs `clear; clc`. It draws one figure in case 3.

Logic. All cases share a setup of 2000 observations, 8 unknowns and per-row sigma in [0.5, 1.5], seeded with 2026. Each case fits an in-memory `struct('A',A,'l',l,'p',p)`:

1. **Clean:** the baseline.

2. **Outliers:** three gross errors at rows 137, 892 and 1455. The case prints `R2.diag.flagIdx` and a note on rejecting only the largest `|w|` at a time.
3. **Systematic error:** an unmodelled `1.2 sin(2 pi t)`. The case prints `nFlag` and plots `R3.diag.w` against time with the `±kCrit` lines, above the clean case for comparison.
4. **Uncontrolled observation:** unknown 8 is seen only by observation 500. The case prints `nFlag` and the redundancy number `rNum(500)`.
5. **Wrong weights:** group 2 is three times noisier than claimed. The case prints `varRatio` and `nFlag`, then shows the cure: a streaming declaration with two observation types `g1` and `g2` (local helper `groupDay`) and `Estimate="noise", Pool="stream", ArcFactors=false`. It prints the recovered sigma ratio (truth 3) and `sigma0`.
6. **Collinear parameters:** column 2 is almost column 1. The case prints `R6.diag.corrMax`.

Output. A closing summary prints one line per case with `R.report('Verbose', false).verdict`.

11.9 `demo_noisePolicy`: what you know about each stream

Purpose. This example shows the per-stream policy fields of a covariance spec. `estimate` is `"none"`, `"scale"` or `"law"`, and `arcFactors` is `true` or `false`. Each choice recovers a planted truth. For the policy semantics, see the chapter *Choosing the stochastic model*.

Run.

```
[R, s] = demo_noisePolicy();
[R, s] = demo_noisePolicy('maxPasses', 20, 'verbose', true);   % defaults: 10, false
```

`R` is the `lsqvce.Results`. `s` summarises what was measured:

- `.stream`, `.declared`, `.level` and `.truth`, per stream;
- `.noneFixed`;
- `.rho1`: the estimated and planted lag-1 correlation of `rr`;
- `.arcFactor` and `.arcRatio`;
- `.passes`;
- `.badEstimate`: the identifier that a misspelt policy raises.

The case. The data have one system, three days and three arcs a day of 256 epochs (5 s), four common harmonic coefficients, and one `'arc'`-scoped bias. There are four streams:

Stream	Truth	Declared	Policy
`gps`	white, sigma 0.2	`diag`, 0.2 (correct)	`"none"`
`acc`	AR(1), phi 0.85, level 0.5	`psd` of the same filter, sigma 1	`"scale"`
`rr`	AR(1), phi 0.5	`diag`, 1 (white)	`"law"`
`mag`	white 0.3, arc 2 three times worse	`diag`, 0.3	`"scale"`, `arcFactors = true`

Implementation.

- `policyData` builds the declaration. For `acc` it builds a `'psd'` law from `firstOrderPsd`, which gives the one-sided PSD on a frequency grid.
- `policyDay` is the six-argument `buildDay`. It uses `info.nl` and `info.offsetOf(a)` to place the bias column, and it returns one cell per arc.
- `policyNoise` draws the planted noise from a stream seeded on day and stream, so that `buildDay` is deterministic.
- All four policies run in one call with `Options(Estimate="noise", Pool="stream", ArcFactors=false, MaxPasses=o.maxPasses, Noise=struct('Smooth', 1))`. `ArcFactors=false` is the run default. `mag` overrides it on its own spec. The loop converges in 3 passes. Without the smoothing, the 60-bin law of `rr` needed 14 passes and settled on the same values to three digits.

Output.

- A table of the level per stream (declared, estimated, planted) from `R.noise.model{k}.sigma`, with streams looked up by name.
- The lag-1 correlation of `rr` from `R.noise.Cgroups`.
- The mean arc factor per arc of `mag` from `R.noise.model{k}.sigma2`, and the ratio of the bad arc to the good arcs (planted: 9).
- The identifier with which `fit` refuses `estimate = "shape"`.
- `R.summary()`.

The help block records the measured values of one run. For example, `gps` keeps its declared 0.2 exactly, and the bad-arc variance ratio is 9.45 against the planted 9. Only the ratio between arc factors is identifiable, because the level absorbs their mean.

11.10 `demo_priors`: regularisation in the announced layout

Purpose. In a scoped declaration you do not know in advance which column of `A` or `B` an unknown ends up in. `lsqvce.layout(data)` announces the layout before `fit` runs, so you can write a prior operator `L` in those columns. For the full reference, see the chapter *Priors and regularisation*.

Run.

```
[R, s] = demo_priors();
[R, s] = demo_priors('verbose', true);
```

`R` is the regularised solve. `s` holds:

- `.nc` and `.nl`;
- `.nullDim`;
- `.unregularised`: the error identifier;
- `.sigma0` and `.zSignal`;
- `.nPseudo`, `.formGap`, `.datumGap` and `.priorLevel`.

The case. The data have four harmonic signal coefficients (`'common'`) and one offset per day (a `'day'` row of `commonParams`). The system has two `'arc'` rows (bias, drift) and one `'system'` row (a scale on a known auxiliary series). There are three days with two arcs of 150 epochs each. The day offset and the arc biases cannot be separated, so the normal matrix has one null direction per day.

Logic.

1. Call `lay = lsqvce.layout(data)`. Print `lay.nc`, `lay.systems(1).nl` and the column map from `lay.common` and `lay.systems(1).local`, and find the day-offset columns with `lay.colCommon(row, day)`.
2. Count the null directions on a densely assembled `[A B]` (`rank`). Fit without priors and record the refusal identifier.
3. Declare two common groups in `data.priors`: a tight `'datum'` (the offsets average to zero) and a loose `'smooth'` (first differences), both with `estimate "none"`.
4. Declare a local group `'drift'` on `data.systems(1).priors`, written once through `lay.systems(1).col(row, day, arc)` and once as the pattern `group = '^drift$'`. Both solutions are compared.
5. Move the datum (`x0 = 0.5`) and measure that the signal stays put while the offsets move.
6. Run `Options(Estimate="noise", Pool="stream", ArcFactors=false, MaxPasses=4)` with the drift group at `estimate "scale"`. The group appears as its own stream `x1|prior_drift`, and its level is printed.

Implementation.

- `priorData` builds the declaration.
- `priorDay` is the six-argument `buildDay`. It uses `info.commonDayOffset`, `info.sysOffset` and `info.offsetOf(a)`.

- `nullDimension` and `layoutInfo` assemble the dense rank check. That check is for demonstration only; in your own code `fit` supplies `info`.
- `fitWithoutGroups` gives the pseudo-observation count.
- `R.coefficients().se` gives the z-scores of the signal.

11.11 `demo_combine`: several systems by elimination

Purpose. This example shows how to declare several systems that share common parameters, where each system also has local parameters that only it observes. `lsqvce.fit` eliminates the locals system by system. The example also shows how to check the degrees of freedom. A parameter is local only if exactly one data set observes it.

Run. `demo_combine()` takes no arguments and returns nothing. It runs `clc`.

Logic and implementation.

- The data have six common unknowns and three systems with 12, 8 and 15 `'system'`-scoped locals. The systems have 40, 30 and 50 days of 25 observations each, with unit noise, seeded with 2026.
- Each `data.systems(i)` has:
  - `buildDay`: from `makeBuildDay`, which reads the pre-generated rows, one arc per day;
  - `covariance.obs = struct('type','diag','sigma',1);`
  - `localParams`;
  - its own `days`;
  - `name`.
- One call `lsqvce.fit(data, lsqvce.Options(Verbose=false))`.

Output.

- An integrity block that compares `n - u_total` (the eliminated locals included) with `R.diag.r`.
- A table of the common parameters: estimate, truth, error, and sigma from `R.sol.Cxx`.
- `R.sigma0`, `R.diag.varRatio` and `R.report`.
- A cost block that compares the full and the reduced normal-matrix sizes.

11.12 `demo_sensors`: system / sensor / day, gaps, and a bad day

Purpose. This example declares the full hierarchy:

- a system is an entry of `data.systems`;
- a sensor is an observation type, one field of `covariance`;
- a day is an entry of `data.days`.

It shows a data gap, finds a bad day through one variance factor per sensor-day, and then estimates the per-sensor noise levels without that day. The order is the point: a variance component is an average over its stream, so an event inside the stream corrupts the level.

Run. `demo_sensors()` takes no arguments and returns nothing.

The case. There are six common unknowns and two systems:

- `GRACE-C` (`id 'C'`) carries `KBR`, `LRI` and `GPS` and has 4 locals;
- `GRACE-D` (`id 'D'`) carries `KBR` and `LRI` and has 3 locals.

The data cover 10 days with 60 observations per sensor and day. The true sigmas are KBR 1.0, LRI 0.4 and GPS 2.5, but every sensor is declared with sigma 1. GRACE-D's LRI is missing on days 4–5, and GRACE-C's KBR carries a 6-sigma offset on day 7.

Logic.

1. Generate the data into a `containers.Map` per system.
2. Declare the data. `s.arcs` is a resolver from `makeSchedule` that returns no arcs on an outage day, which is how a data gap is declared.

3. Solve once with `Options(Verbose=false)` and print `R.sigma0` (the symptom).
4. Find the bad day. Run `Options(Estimate="noise", Pool="obsType", ArcFactors=true, MaxPasses=20, Noise=struct('Grid', 8))`. Every arc, here one system, sensor and day, gets its own variance factor. The demo looks each factor up by `lsqvce.util.arcKey` in `R.noise.arcKeys` and prints it divided by the median of its stream, because the factors are absolute against the claimed sigma 1. An arc above 5 times its stream's median is marked `<<< EVENT`.
5. Remove each event's arc from the store, so the arc resolver returns no arc for it, as for an outage. Then run `Options(Estimate="noise", Pool="obsType", ArcFactors=false, Noise=struct('Grid', 8))`. The KBR streams of both systems share one spectral shape, and every stream keeps its own level. The demo prints `R.summary().sigma` per stream against its truth.

`Noise.Grid 8` is used because an arc holds only 60 samples. With the default 60-bin law, the bins that the data barely determine decay toward zero, and the loop needed 168 passes. The help block records the measured values: the event is 15.9 times its stream's median, every other sensor-day lies between 0.5 and 1.8, and the levels come back within 4.0 % of the truth. Estimated with the event still in the data, C|KBR came back 46 % high.

11.13 `demo_grace` and `demo_grace_dataLoader`: the noise loop on multi-axis streams

Purpose. This is a complete `Estimate="noise"` run on a GRACE-FO-shaped synthetic problem with three-axis position streams and a scalar inter-satellite stream. Every stream starts from a deliberately wrong a-priori model: white noise with sigma 1. The help text calls it the copy-paste starting point. To use it on your data, replace the loader with your own `buildDay` and `data`.

Run.

```
R = demo_grace();                                    % maxPasses 10, verbose true
R = demo_grace('maxPasses', 8, 'verbose', false);
[cfg, data] = demo_grace_dataLoader();               % the declaration alone
```

Outline. `demo_grace` holds the run and the reading. `demo_grace_dataLoader` holds the declaration, with the local functions:

- `graceData`, `demoCfg` and `whiteCov`;
- `demoBuildDay`, the five-argument form `(day, obsType, common, local, procOpt)`;
- `commonParamTable` and `localParamTable`;
- the generators `A_design`, `B_design` and `colouredNoise`.

The declaration. There are two systems:

- **System 1** merges satellites C and D. It carries the streams `posC`, `posD` and `rr`. The `rr` design couples the arc states of both satellites, and one system has only one local block, so C and D are merged with a stacked local block `[y_C | y_D]`.
- **System 2** is satellite E with `posE` only.

Both systems share six common unknowns. The data cover four days with two arcs a day and 300 epochs per arc (5 s). The planted sigmas are 0.8, 1.3, 1.6 and 1.1.

The parameter tables have 15 columns. Column 1 is the scope token (`'common'` or `'system'`), and the other 14 follow this demo's own convention, listed in the loader's help. `processingOptions` is passed as the fifth argument.

For the position streams, `buildDay` returns the per-epoch 3×3 covariance as `D.Cep`, and the toolbox combines it with the temporal law. Rows are in epoch-major order. `buildDay` returns one cell per arc.

Run options and output. The run uses `Options(Estimate="noise", Pool="stream", ArcFactors=false, MaxPasses=o.maxPasses, Tolerance=1e-3, Noise=struct('Smooth', 1))`.

`Tolerance=1e-3` shows the settled values in more digits. With `Noise.Smooth 1` the loop converges in 4 passes. The plain 60-bin law needed 27 passes and settled on the same levels. The demo prints:

- the true sigmas;
- `R.history.sigma`, one row per iteration and one column per stream;
- `R.summary()`, one row per stream;
- `R.convergence()`, one row per iteration;
- `R.report()`;
- `R.converged` and `R.history.nIter`.

The estimated shape is available through `R.plot("psd")` and `R.noise.S`. In `R.summary()` the `arOrder` column stays `NaN`, because the estimator fits a spectrum and not an AR order.

11.14 `demo_gravity`, `demo_gravity_dataLoader` and `demo_gravity_design`: a physically structured gravity recovery

Purpose. This example runs the same noise loop on a design whose columns are real spherical-harmonic potential partials along a near-polar orbit, so the normal matrix has order-block correlation, polar-gap conditioning and degree attenuation. It also shows how to read a gravity solution in units of its own formal error.

Run.

```
R = demo_gravity();
[R, score] = demo_gravity('MaxPasses', 6, 'Verbose', true);
R = demo_gravity('Plots', true, 'ExportDir', '/tmp/figs');   % two figures, saved as PNG
```

The name-value options are:

- `MaxPasses` (default 10; the run converges in 4);
- `Verbose` (default `false`);
- `Plots` (default `false`);
- `ExportDir` (default `''`).

`score` contains:

- `.sigma0`;
- `.stream`, `.sigmaPlanted`, `.sigmaRecovered` and `.sigmaRatio`;
- `.zMax` and `.zMeanSq`;
- `.anomaly` (`.true`, `.est`, `.sigma`, `.z`);
- `.degree` and `.degRatio`.

An `ExportDir` that does not exist is an error. The help states a cost of about 1.6 s: 5 days, 16 arcs, 3 streams, 77 common and 96 local unknowns.

Outline. There are three files:

- `demo_gravity` runs the problem and reads it. Its local functions are `section`, `streamType`, `degreeVariances`, `plotDegreeVariances`, `plotCoefficientErrors` and `exportIf`.
- `demo_gravity_dataLoader()` returns `[cfg, data]`. Its local functions are `gravityData`, `gravityCfg`, `buildGravityDay` and `arcOrbit`.
- `demo_gravity_design(nmax, orbit)` returns `[A, meta]`. `A` has `(nmax+1)^2 - 4` columns (degrees 2..nmax, C and S terms). `meta.n`, `meta.m` and `meta.cs` label the columns. `orbit` needs the fields `u`, `lam0`, `incl` and `r` (with `r > 1`); bad input raises an error that names the faulty field.

The declaration. The field goes to degree `nmax = 8`, which gives 77 coefficients. The planted field follows a closed formula plus one anomaly at (5, 3, C). `commonParams` has four columns: the scope token, `n`, `m` and the C/S term.

There are three systems:

System	Stream	Locals per arc
GRACE-C	potC	2
GRACE-D	potD, trailing C along-track	3
KBR	kbr: the potential difference D − C, whose design couples both orbits	1

The locals are 'arc'-scoped. The arc schedule is ragged: 3, 4, 2, 4 and 3 arcs per day. Arcs are missing by ID on days 3 and 5, and every arc has its own length between 140 and 220 epochs. Because of the 'arc' rows, the schedule is declared as a resolver, arcs = @(day, obsType) cfg.arcsOfDay{...}.

The declared covariance has the right AR shape and unit sigma. The planted sigmas are 1.0, 2.0 and 0.5.

buildGravityDay is the six-argument form. It reads arcInfo.nl and arcInfo.offsetOf(a) and returns one cell per scheduled arc.

Run options and output. The run uses Options(Estimate="noise", Pool="stream", ArcFactors=false, MaxPasses=opt.MaxPasses, Noise=struct('Smooth', 1)). It converges in 4 passes. The per-arc locals absorb the zero-frequency bin of the 60-bin law, so without the smoothing that bin settled slowly and the loop needed 72 passes. The demo prints three sections:

1. **The problem:** sizes, schedule and local counts.
2. **The weights:** planted and recovered sigma per stream from R.noise.model{k}.sigma, with their ratio. Only the ratios matter, because a common factor goes into sigma0.
3. **The field:** z = (R.x - xTrue) ./ sqrt(diag(R.sol.Cxx)) (R.sol.Cxx is scaled by σ_0^2 already), max |z|, mean z^2, the anomaly with its sigma, and the ratio of the error degree variance to its formal value per degree.

The help block records the numbers of one measured run.

11.15 demo_gravfire and template/gravfire_data.m: a GRAVFIRE bundle

Purpose. This example maps a GRAVFIRE preprocessing bundle (commonData, myOptions, systemsData) onto data and runs it through lsqvce.fit. The bundle contains:

- Stokes coefficients as the common block;
- per-day initial conditions, accelerometer parameters and empirical parameters as locals;
- kinematic positions of two satellites and the K-band range-rate as observation groups.

The design matrices are not part of the bundle. You supply them.

Run.

```
R = demo_gravfire(bundleFile);                                  % Estimate "none"
R = demo_gravfire(bundleFile, Estimate="noise", Passes=3, Verbose=1);
```

- bundleFile defaults to "", which means testdata.mat in the folder above the toolbox root.
- Estimate is "none" (default, one solve) or "noise".
- Passes is the MaxPasses of the noise run (default 3).
- Verbose is 0..2 (default 1).

The bundle is private and not shipped. If the file does not exist, the demo prints what it would do and returns [].

Logic. With a bundle, the demo:

1. adds template/ to the path and loads the file;
2. calls gravfire_data(..., 'DryRun', true);

3. runs `lsqvce.fit(data, Verbose=...)`, or for `"noise"` `lsqvce.fit(data, Estimate="noise", Pool="obsType", Noise=struct('Grid', 2160), MaxPasses=Passes, CacheDays=Inf, OnPass=watch)`, with the not-converged warning suppressed;
4. prints `R.report` when `Verbose > 0`.

`fit` accepts the options as name-value pairs here, without an `lsqvce.Options` object. `DryRun` puts random design matrices in place of the real design step, so the demo tests the pipeline, not gravity recovery.

`gravfire_data(commonData, myOptions, systemsData, Name=Value).` The name-value options are:

- `'DesignFcn'`: your function `[A, B] = f(o, day, obsType, common, local, po, info)`, returning `m × info.nc` and `m × info.nl` with rows in epoch-major order;
- `'DryRun'`: deterministic random `A` and `B`, seeded per day and observation type.

The function returns a `data` struct with `commonParams`, `days`, `priors` and one system (`id 1`, `name 'GRACE'`) with `buildDay`, `covariance`, `localParams`, `priors` and `processingOptions`. It declares no arc schedule: the pieces that `buildDay` returns as cells are the arcs.

The comments are numbered as steps:

1. Prefix the 14-column common table with `'common'`.
2. Take the calendar from `myOptions.days`.
3. Name the observation types: `posA`/`posB` for `kinematicPosition`, `kbr` for `range-rate`. Each gets a white `diag` with `sigma = sqrt(variance(g))`.
4. Take one day's rows of the expanded local table as `'day'`-scoped templates, with times in seconds of the day.
5. (No arc schedule.)
6. Bind everything `buildDay` needs into `processingOptions`.
7. Assemble the system.

The local function `buildGravfireDay` does the following:

- drops flagged epochs;
- builds epoch-major `l` and `t`;
- converts `iSpp` into a per-epoch 3×3 `Cep`;
- calls the design step;
- cuts the day into one cell per piece.

Regularisation comes from column 11 of the tables: `{'Tikhonov', fromPass, sigma[, label]}`. On common rows, `priorsFromColumn11` builds `data.priors` in the `L` form. On local rows, `gravfireLocalPriors` builds `data.systems.priors` in the `group` (regular-expression) form. Labelled groups get `estimate "scale"`; unlabelled rows get `"none"`. The rows of one label must share sigma and `fromPass`, and on local rows also the a-priori value (column 10): the `group` form expands its rows in the toolbox's order, so one value per member cannot be written there. The test suite runs the whole template on a generated bundle (`dev/tests/fixture_gravfire_bundle.m`).

The function raises these errors:

Identifier	Cause
`gravfire:designMissing`	no `DesignFcn` and no `DryRun`, raised when a day is read
`gravfire:unknownQuantity`	an observation group has an unsupported quantity
`gravfire:designShape`	`DesignFcn` returned `A` or `B` with the wrong size
`gravfire:badConstraint`	a malformed column-11 entry, or rows of one label that differ in sigma, `fromPass` or (local rows) the a-priori value

The help also refers to `dev/GRAVFIRE_INTERFACE.md`, the step-by-step guide.

11.16 `template/kbr_variational.m`: GRACE-FO range-rates by the variational equations

Purpose. This is a real-data template. It turns GRACE-FO Level-1B data into a `data` declaration for a gravity field in fully normalised spherical harmonics:

- range-rates (KBR1B) of the C/D pair;
- reduced-dynamic positions (GNV1B);
- accelerometer (ACT1B) and attitude (SCA1B) data.

For each arc and satellite, the reference orbit and the state transition `Psi` are integrated together with an 8th-order Adams–Bashforth–Moulton integrator, started with RK4. The acceleration, its gradient and the coefficient partials are analytic. The range-rate rows are formed on the fly, so `Psi` is never stored.

Run.

```
cfg = struct('days', 60000:60029, 'dataDir', '/data/gracefo/L1B', ...
             'gravityField', '/data/models/GOCO06s.gfc', ...
             'trf2crf', @myIers2010, 'degree', [2 60], 'arcsPerDay', 4);
[data, kit] = kbr_variational(cfg);
R  = lsqvce.fit(data, lsqvce.Options(Estimate="noise", Noise=struct('Grid', 1080)));
dC = kit.unitCommon .* R.x;                          % physical corrections
cfg.fieldUpdate = dC;  data = kbr_variational(cfg);    % second linearisation
```

Configuration. Four fields of `cfg` are required:

- `days` (MJD);
- `dataDir`;
- `gravityField`: an ICGEM `.gfc` file or a struct `.GM .R .C .S`;
- `trf2crf`: a function `[R, Rdot] = f(tGps)`.

The other fields default as follows:

Field	Default
`degree`	`[2 60]`
`fieldDegree`	`degree(2)`
`arcsPerDay`	1
`arcs`	none; give a resolver to schedule arcs
`step`	5 s
`posSampling`	30 s
`sigma.kbr`	2e-7
`sigma.pos`	0.02
`obsTypes`	`{'kbr','posC','posD'}`
`files, formats`	GRACE-FO Level-1B v04 ASCII
`quatConvention`	`'srf2crf'`
`accToSrf`	`eye(3)`
`accBias0`	0
`accScale0`	1
`fieldUpdate`	none
`background`	no other forces
`maxAccGap`	10 s

Outputs. `data` has:

- `commonParams`: C_nm and S_nm with a unit of 1e-10 in column 5;

- days;
- one system (id 1, 'GRACE-FO') with:
  - 'arc' rows: initial position and velocity per satellite, and the KBR bias, drift and once-per-revolution cos/sin terms;
  - 'day' rows: accelerometer bias and scale per satellite;
  - arcs, covariance, processingOptions, prepareDay and buildDay.

Every column carries a declared unit, so the physical value is unit .* estimate. Position streams carry their sigmas in Cep.

kit exposes the physics as function handles:

- unitCommon and unitLocal;
- coeffIndex, harmonics, makeField and fieldEval;
- propagate and accModel;
- eraRotation, readGfc, readL1B, prepareDay and buildDay.

prepareKbrDay, the system's prepareDay, integrates the whole day once each time the day is built and forms the rows of every stream; buildKbrDay picks its stream out of procOpt.prepared. Only the current day's integration is kept; CacheDays keeps the built rows. Called directly, kit.buildDay needs procOpt.prepared set to what kit.prepareDay returns for the day.

Limitations. The help lists these:

- kit.eraRotation is a stand-in for synthetic data only.
- L1B layouts and frame conventions must be checked against the product handbook.
- Forces other than the static field and the accelerometer enter only through background.
- KBR quality flags are not screened.
- Memory per stream is 8 m nc bytes per day, about 0.5 GB for KBR at 5 s and degree 60.

Errors carry kbrvar: identifiers, for example kbrvar:badConfig, kbrvar:noFile, kbrvar:accGap and kbrvar:emptyArc.

11.17 template/lsqvce_template.m: the engine-tier starting point

Purpose. This template builds the cell array of Loader objects that the engine-tier functions take (lsqvce.engine.adjustBlock, lsqvce.engine.adjustRobust). Use it when your pipeline is driven by a procOpt configuration struct and you want the Loader objects in hand. On the user path you do not need it: declare data and call lsqvce.fit.

Run. Copy the file, rename it, and write the stub myBuildDay:

```
L = myCase(procOpt);
R = lsqvce.engine.adjustBlock(L, procOpt.days);
R.report();
```

Structure.

- **buildLoaders** is real code. It makes one lsqvce.engine.makeLoader per entry of procOpt.observationGroups, with:
  - the observation types;
  - a white, unit a-priori covariance (aprioriCovariance);
  - the five-argument buildDay;
  - systemId, which must be numeric or a single character;
  - days, commonParams, localParams and processingOptions.
- **myBuildDay** is a stub. It raises lsqvce_template:notImplemented until you replace it. It must be deterministic and return A, B, l, t and optionally Cep as one cell per arc, with multi-axis rows in epoch-major order.
- A procOpt without observationGroups or days raises lsqvce_template:badProcOpt.

11.18 `demo_messaging`: what each verbosity level prints

Purpose. This example lets you compare the verbosity levels on real output before you adopt a level for long runs. See also the chapter *Watching a run*.

Run.

```
demo_messaging()                          % levels [0 1 2]
demo_messaging('levels', [0 1])
demo_messaging('toFile', '/tmp/levels.txt')
```

There are no outputs; everything is printed. A file that cannot be opened raises `demo_messaging:badFile`.

Logic. For each level, `runAtLevel` redirects the toolbox log with `lsqvce.util.log.set('fid', fid, 'screen', false)` and builds `demo_grace_dataLoader()`. It runs `Options(Estimate="noise", Pool="stream", ArcFactors=false, Verbose=lvl, MaxPasses=1)`, then reads the captured text and prints it under a header. The header gives the level and `lsqvce.util.verbosity.describe(lvl)`.

What you will see.

- **Level 0** prints nothing but warnings. Warnings are not controlled by the verbosity level, and a single pass does not converge.
- **Level 1** prints the stage label for each swept system and a convergence line for each pass.
- **Level 2** prints the same as level 1 when you go through `lsqvce.fit`. The help block explains why.

The per-stream view is not printed. Read it afterwards with `R.summary()` or `R.convergence('Plot', true)`.

11.19 `bench_qr_vs_normal`: which backend, measured on your machine

Purpose. This benchmark measures the cost of `Backend="qr"` against `Backend="normal"` on your hardware, and shows when the accuracy of QR is needed.

Run.

```
bench_qr_vs_normal()                                        % u = [10 30 50 100], n = 1e5, nk = 50
bench_qr_vs_normal('u', [10 50 100], 'n', 2e5, 'nk', 5000)
```

There are no outputs; the benchmark prints two tables.

Logic. Both runs use the same batched declaration, `struct('batchFcn', src, 'numUnknowns', u, 'numBatches', nb)`, with `Options(Backend="normal")` and `Options(Backend="qr")`.

1. **Timing.** For each `u`, the benchmark times both runs, keeping the best of `reps` repetitions. It prints the times, their ratio, and the relative agreement of `R.x`. The timed call is the whole `fit`, including diagnostics, so the difference between the two times is the cost of the backend.
2. **Accuracy against conditioning.** With `u = 10` and `n = 20000`, the benchmark makes column 2 nearly collinear with column 1, for `eps = 1e-1 … 1e-8`. It compares both backends with an SVD reference and prints `cond(R.sol.Cxx)`, which equals `cond(N)`, the two errors, and a verdict. When `N` is singular in double precision, the normal path refuses with `lsqvce:normalEq:notSPD`. The benchmark catches that refusal and reports QR as mandatory.

The recipe the benchmark prints: solve once with `"normal"`, read `cond(R.sol.Cxx)`, and switch to `"qr"` only when that number calls for it. For the backend option itself, see the chapter *`lsqvce.Options`* of the API reference.

12 Large problems

A GRACE-FO month at degree 60 has millions of observations, about 3700 common and tens of thousands of local parameters. This chapter lists the settings that decide whether such a run fits in time and memory. The defaults are safe; change them when you have measured.

12.1 Memory

The accumulator holds the reduced common system, nc × nc. Local parameters are eliminated arc by arc and never enter it. The largest transient object is one built day: m × nc doubles per observation type — at degree 60 about 190 MB for a six-hour three-axis arc.

`CacheDays` (default 1) is the number of built days kept resident per system. An iterative run reads every day twice per pass; `CacheDays = Inf` builds each day once per run if the calendar fits in memory. A system's `prepareDay` result is kept for the current day only, whatever `CacheDays`: it adds one day's worth, and runs again whenever a day is rebuilt. If reading the raw files is the expensive part, cache those inside your `buildDay` closure instead.

12.2 Time

setting	effect
`Noise.Whitener = "ar"`	whitens a long arc in O(m·p) instead of the O(m³) of the dense Toeplitz factor, which is formed over the whole arc (m² memory, refused above `maxDenseArc` = 20 000 epochs); check the AR fit once with `lsqvce.engine.checkPSD`. It speeds up the SOLVE: under `Estimate = "noise"` the estimator's sweep still forms every arc's covariance densely (m² memory, O(m³) per arc and pass), and the same `maxDenseArc` refuses a longer arc there — cut long arcs, or estimate the law on a subset and solve the whole with `Estimate = "none"`
`Probes`	the cost of the redundancy sweep grows linearly with it; 100 is the standard, fewer gives noisier redundancies
`Parallel = struct('Mode', "parfor", 'DistributeDays', true)`	distributes the day loop of the accumulation over workers; the result is bitwise identical to the serial run
`MaxPasses`, `Tolerance`	the number of passes of an iterative run

12.3 Resampling days

A bootstrap over days refits the problem once per replicate, and each refit re-reads and re-eliminates days the first fit already reduced. `R.contributions()` keeps every day's reduced contribution (one more pass over the data), and `lsqvce.solveContributions(C, c)` solves a replicate from them in O(D·nc²): `c` says how often the replicate drew each day. The days keep the run's final weights; the noise model and robust weights are not re-estimated per replicate. Declare day-specific unknowns per day or per arc: a local that spans days couples them, and the days then have no separate contributions.

12.4 Backend

`Backend = "qr"` (default) keeps the square-root information form and is the safer choice on ill-conditioned common blocks; `"normal"` forms the normal equations and is faster on well-conditioned ones. Both give the same numbers to rounding where both work. `examples/bench_qr_vs_normal.m` measures the trade-off on your machine.

12.5 Before a long run

1. Run one day with `Estimate = "none"` and read the report.
2. Run a few days with `Estimate = "noise"` and `MaxPasses = 3` and look at `R.plot("psd")` and `R.summary()`.
3. Fix `Seed` so the long run is reproducible, and save `R` with its `meta`.

13 Troubleshooting

Every error the toolbox raises has an identifier (`ME.identifier`) and a message that names what is wrong: the field, the system, the day, the stream. Read the message first; this chapter adds the usual cause and the usual fix. The full list of identifiers is in the API reference (chapter *Error and warning identifiers*).

```
try
    R = lsqvce.fit(data, opts);
catch ME
    disp(ME.identifier)
    disp(ME.message)
    if ~isempty(ME.cause), disp(ME.cause{1}.message), end   % YOUR error, when your function thre
end
```

13.1 Before any data is read: the declaration

`fit` checks the whole declaration before it calls your functions. These errors cost nothing to fix and to re-run. A few of them are raised while `fit` builds the loaders; `fit` reports those under its own prefix: the table lists them by the `makeLoader` identifier, and the `ME.identifier` you see is the prefix `lsqvce:fit:` followed by its last part (`prepareUnused`, say), with the original attached as the cause.

identifier	usual cause	fix
`lsqvce:fit:unknownField`	a typo in a field name, e.g. `covaraince`; the message lists the allowed names	correct the name. Unknown fields are errors on purpose: a typo is never ignored
`lsqvce:fit:missingField`	a system without `buildDay` or without `covariance`	add it
`lsqvce:fit:noCommon`	`data.commonParams` missing, or not a cell table	one row per parameter, scope token in column 1
`lsqvce:fit:badScope`	a scope token that does not exist (`'arcs'` instead of `'arc'`)	`commonParams`: `'common'` or `'day'`; `localParams`: `'arc'`, `'day'` or `'system'`
`lsqvce:fit:noDays`, `badDays`, `duplicateDay`	no day list, a non-integer day, or a day listed twice	`data.days = 1:30` (or a system's own days)
`lsqvce:fit:builderKind`	`buildDay` or `prepareDay` is a name (`'myBuildDay'`) instead of a handle	`@myBuildDay`

identifier	usual cause	fix
`lsqvce:makeLoader:localPlan`	`'arc'` parameters without an arc schedule	declare `s.arcs` (a count, an id vector or `@(day, obsType)`)
`lsqvce:makeLoader:prepareUnused`	`prepareDay` with a `buildDay` that declares fewer than five arguments, so it could never see the result	declare `buildDay(day, obsType, common, local, procOpt[, info])`
`lsqvce:makeLoader:preparedField`	your `processingOptions` already has a field `prepared`	rename your field; the name is taken by `prepareDay`'s result
`lsqvce:fit:duplicateSystem`	two systems with the same `id`	give each system its own `id`
`lsqvce:fit:badPrior`	`L` not `p x nc`, or `x0` of the wrong length	write the prior in the columns `lsqvce.layout(data)` announces
`lsqvce:fit:modelKind`	`Estimate = "noise"` on the matrices or batched kind	the noise estimation needs streams: use the streaming kind

13.2 While reading: your `buildDay` and `prepareDay`

These come from inside the run, on the first day where something is wrong. The message names the day and the stream.

identifier	usual cause	fix
`lsqvce:makeLoader:buildDay`	your `buildDay` threw an error	read `ME.cause{1}`: it is your own error, unchanged. Call your `buildDay` for that day by hand
`lsqvce:makeLoader:prepareDay`	your `prepareDay` threw an error	as above, with `prepareDay`
`lsqvce:makeLoader:dayShape`	`A`, `B`, `l`, `t` are not cells of one per arc, or the arcs disagree in their numbers of rows	return `struct('A', {cellOfArcs}, ...)`: note the braces around the cell
`lsqvce:accum:arcBadNc`, `arcBadNl`	`A` does not have `info.nc` columns, or `B` not `info.nl`	build `B = zeros(m, info.nl)` and place each block by `info.offsetOf`, `info.dayOffset`
`lsqvce:covModel:denseArcTooLong`	an arc longer than 20 000 epochs under a dense noise law or under `Estimate = "noise"`	cut the arcs shorter, or see *Large problems*

A frequent mistake that raises nothing: a `buildDay` that is **not deterministic**, for example one drawing fresh random noise on every call. The toolbox reads each day at least twice per pass and then sees two different data sets; the report's integrity check (Level 0) is the place where this shows. Seed any random numbers on the day and the stream, as the tutorial does.

13.3 During the solve

message	meaning	what to do
`lsqvce:qr:singularR` / `lsqvce:qr:rankDeficient`	the common parameters are not all determined by the data: a free network without a datum, an offset that a per-arc bias absorbs	add a datum or a regularisation as a prior (*Priors and regularisation*), or remove the parameter the data cannot see. The toolbox never guesses a datum
`lsqvce:adjustVCEjoint:notConverged` (warning)	the noise loop used up `MaxPasses` before it settled; `R.stopReason` is `"maxPasses"`	the last pass is returned in full. Look at `R.plot("convergence")`; raise `MaxPasses`, or see *When the loop did not settle* in *Reading the results and the report*

13.4 The run finishes, but the numbers look wrong

symptom	likely cause	where to read
σ_0 far from 1 in a run with `Estimate = "none"`	the declared `sigma` (or `p`, or `Cep`) is wrong in scale; often an information matrix used where a covariance was expected	*Choosing the stochastic model*, "Run one pass with `Estimate = "none"` first"
σ_0 fine, but the formal errors look too small	the noise is correlated and was declared white	run `Estimate = "noise"`; the tutorial shows the effect
`R.sigma0` is `NaN`	the quadratic form $v^T P v$ cancelled in double precision, and the toolbox reports that instead of clamping it	a `NaN` is a result, not a bug; the theory (section 1.3) explains when it happens
the verdict is `SUSPECT`	at least one finding at `ERROR` level	work through the `[ERROR]` lines of the report, top level first
the verdict is `INVALID`	the integrity check failed; nothing else was evaluated	most often a non-deterministic `buildDay`
`R.summary()` or `R.plot("psd")` raises `lsqvce:Results:noHistory`	the run was a single solve	these need `Estimate = "noise"` (or a robust run)
`R.residuals()` raises `lsqvce:Results:noAccumulator`	the matrices or batched kind	read `R.diag.v` instead

13.5 Asking for help

When you report a problem, include `R.meta` (toolbox version, MATLAB release, the options as resolved, the seeds) or, if `fit` did not return, the full output of `getReport(ME, 'extended')`. Never attach measurement data you are not allowed to share; a case with synthetic data of the same shape is better.